React-Js

React

JavaScript Library

React is a JavaScript library for building user interfaces.

What is React Js?

- React Js is Library developed by Facebook company using the Langauage “JavaScript”.

- ReactJS is a **declarative**, **efficient**, and flexible **JavaScript library** for building reusable UI components. It introduces more efficient and flexible way of building ,organizing and maintaining your UI code.

- It is an open-source, component-based front end library which is responsible only for the view layer of the application.

Some Sparkling Examples of React JS Development

- **Facebook:** The original developer, Facebook's webpage is built using React while it uses React Native for the Android and iOS platforms.
- **Instagram:** Instagram features like Google Maps, Geolocation, search engines etc. are built on React JS features.
- **Netflix:** Credit the app's startup speed, runtime performance, modularity, and other functionalities to React JS.
- **Whatsapp:** The king of instant messaging accredits its UI to React JS.
- **Uber :** Again, the geolocation and google maps integration works on this app due to React JS functionalities.

History-

React was created by **Jordan Walke**, a software engineer at **Facebook-(Meta)**, who released an early prototype of React called "FaxJS".He was influenced by XHP, an HTML component library for PHP. It was first deployed on **Facebook's News Feed in 2011** and later on Instagram in 2012.

- Initial Release to the Public (V-0.3.0) was in July 2013.
- Current version of React.JS -**The current stable version of React JS is 19.2.4, released on January 26, 2026**

React 19.2.4	In React 19, we're adding support for using async functions in transitions to handle pending states, errors, forms, and optimistic updates automatically.
React 18	was released on March 29, 2022
18.0.0	29 March 2022
React 17	was released in August 2020.
React 16	was released in September 2017.

Benefits of Using React-

1.Single-page apps or multi page web apps.-

A single-page application is a browser-based application that does not require the user to reload the page during operation.

Speed and Response Time

Mobile-friendly:

Multi page web applications having multiple pages. These apps load the pages every time user click on various links.

2.Component Based-

It lets you build your application UI by breaking it up into small reusable piece of code called components.

3.React Helps You Build Rich User Interfaces:

React JS enables developers build brilliant, high-quality user interfaces with its declarative components – something that can be an easy recipe to translate business success.

4.React JS is Flexible:

- What we must remember is that React is a library and not exactly a framework. Thus, it can be used on a huge variety of platforms to build scalable user interfaces. React gives the developers the flexibility to integrate it even into existing apps because of its library approach.

5. Virtual DOM: React has provisions of a Virtual DOM (VDOM) which enables easy manipulation. In the event when the state of an object is changed, the VDOM changes that individual object in the real DOM and not all the other existing objects.

6.React Allows SEO Friendly App Development:

An app always ranks high on Google if it has a lower page load time and fast rendering speed.

7.**React Has Extensive Support and Resources:** Since it is developed by Facebook, React is widely and heavily integrated into the Facebook app, website, and as well as Instagram.

Official **website-https://react.dev/**

8.React Has a Community Support.

9.Scope for Testing the Codes

ReactJS applications are extremely easy to test. It offers a scope where the developer can test and debug their codes

10. React Offers Fast Rendering:

An app performance is directly linked to its structure. And DOM structures are extremely hard to manipulate. Thus, Facebook had devised a Virtual DOM structure for React Js, which helps in calculating the risk factor on the VDOM before initiating any real modification

11.JavaScript Syntax Extension (JSX): JSX is used to develop robust user interfaces. Developers can write the HTML structures and JS codes in the same file, which makes understanding and debugging codes an easy feat.

Why we use ReactJS?

- **To develop User Interfaces (UI) that improves the speed of the apps.**
- It uses virtual DOM (JavaScript object), which improves the performance of the app.
- The JavaScript virtual DOM is faster than the regular DOM. We can use ReactJS on the client and server-side as well as with other frameworks.
- It uses component and data patterns that improve readability and helps to maintain larger apps.

❖ Automatic UI updates

❖ Reusable components

❖ Better performance

Disadvantages of real DOM :

- Every time the DOM gets updated, the updated element and its children have to be rendered again to update the UI of our page.

For this, each time there is a component update, the DOM needs to be updated and the UI components have to be re-rendered.

- Recalculating the CSS and changing the layouts involves complex algorithms, and they do affect the performance. So React has a different approach to dealing with this Virtual DOM.

Rendering -Process of describing a user interface based on the application's current state and props.

ReactJS Virtual DOM- React JS Virtual DOM is an in-memory representation of the DOM.

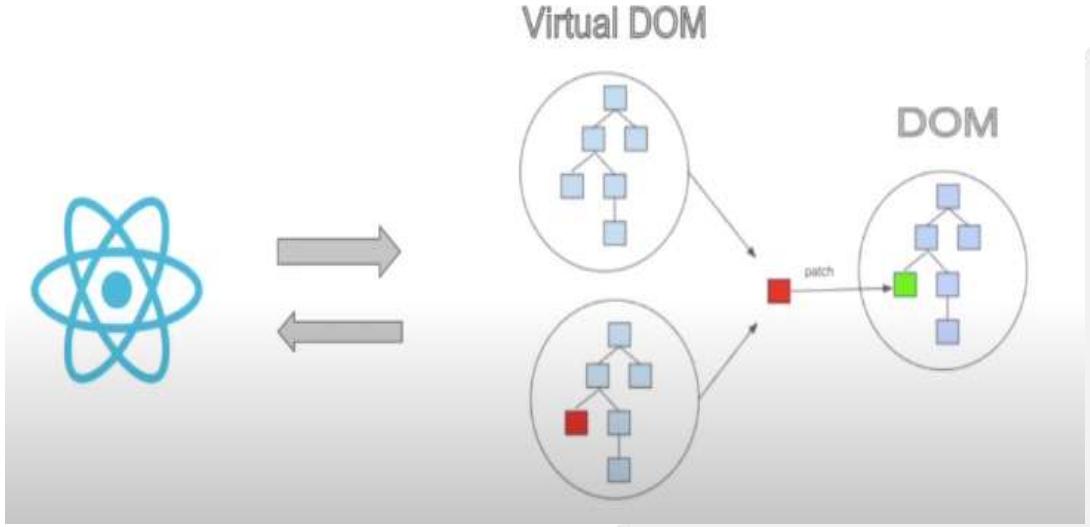
- The virtual DOM (VDOM) is a programming concept where an ideal, or “virtual”, representation of a UI is kept in memory and synced with the “real” DOM by a library such as **ReactDOM**. This process is called reconciliation.

- There is an object for this in React Virtual DOM. It is exactly the same, but it does not have the power to directly change the layout of the document.

- **Manipulating DOM is slow, but manipulating Virtual DOM is fast** as each time there is a change in the state of our application, the virtual DOM gets updated first instead of the real DOM.

Working Process in virtual DOM-

- **Whenever there is a change in the state of any element, a new** Virtual DOM tree is created.
- This new Virtual DOM tree is then compared with the previous Virtual DOM tree and make a note of the changes.
- After this, it finds the best possible ways to make these changes to the real DOM.



- Now only the updated elements will get rendered on the page again.

Differences-

- Real DOM = actual webpage
- Virtual DOM = copy of webpage

Virtual DOM	Real DOM
It is a lightweight copy of the original DOM	It is a tree representation of HTML elements
It is maintained by JavaScript libraries	It is maintained by the browser after parsing HTML elements
After manipulation it only re-renders changed elements	After manipulation, it re-render the entire DOM
Updates are lightweight	Updates are heavyweight
Performance is fast and UX is optimised	Performance is slow and the UX quality is low
Highly efficient as it performs batch updates	Less efficient due to re-rendering of DOM after each update

- Resources
- React Documentation-
- CDN links-https://legacy.reactjs.org/docs/cdn-links.html
- Create React App-https://create-react-app.dev/
- ReactDOM-https://legacy.reactjs.org/docs/react-dom.html
- Babel JavaScript for VS Code
- Set up a local development server
- Live Server VS Code extension-https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer
- http-server-https://www.npmjs.com/package/http-server
- Running a simple local HTTP server-https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Tools_and_setup/set_up_a_local_testing_server#running_a_simple_local_http_server

Add React to a Project- Two common ways to install and get set up with React are

1.Create React app

2.CDN (content delivery network) links.

What Is Create React App (CRA)?

- To compete with frameworks, React has a dedicated framework called Create React App (CRA). It includes file structuring and other tools so that React could be used as a framework. It's the best way to start building a new single-page application in React.

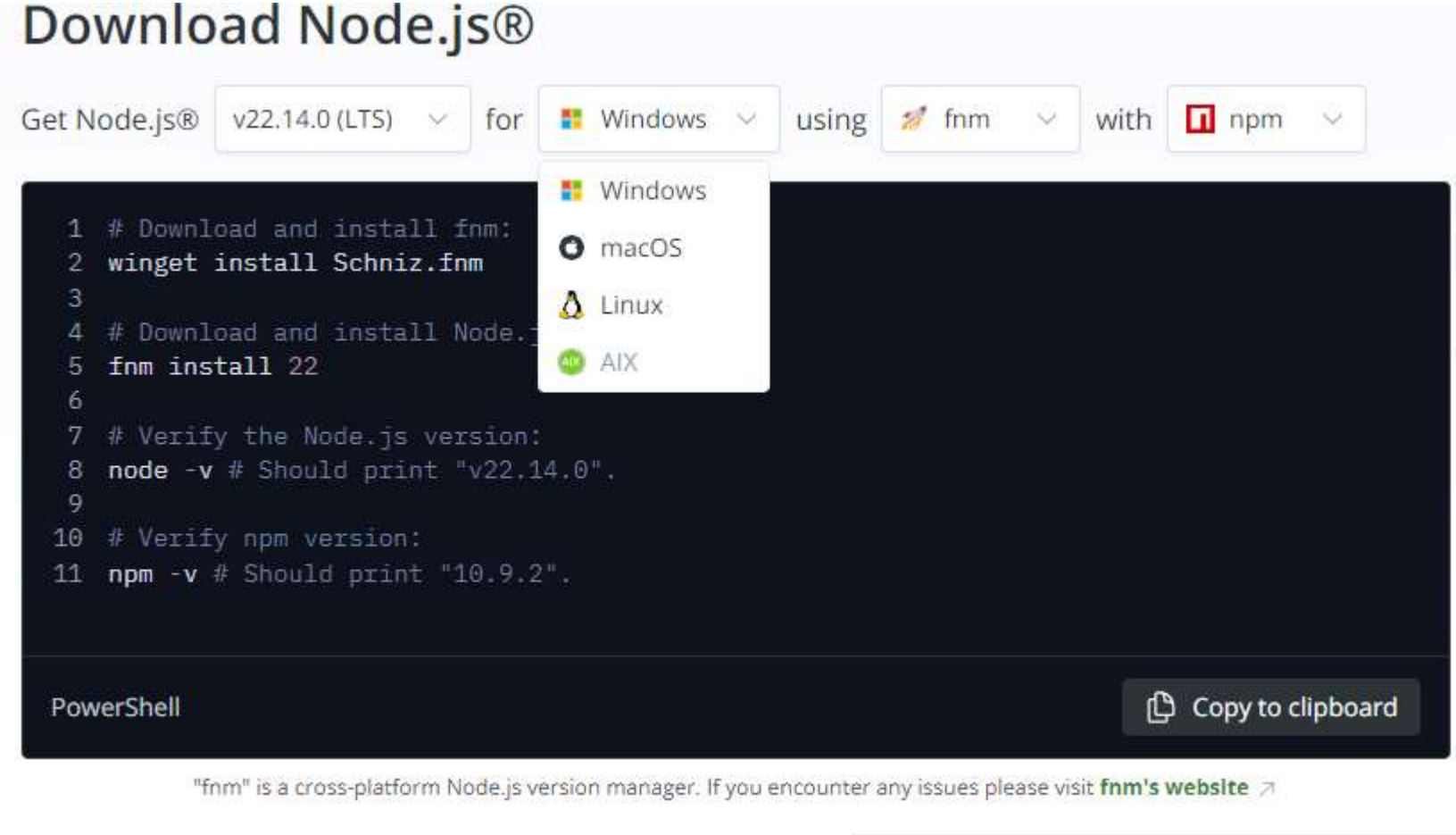
- CRA creates a development environment to build a React-based web app and optimizes your app for production right out of the box. CRA doesn't handle back end logic or databases.

1.Create React App

- Create React App is a comfortable environment for **learning React**, and is the best way to start building **a new single-page application** in React.
- It sets up your development environment so that you can use the latest JavaScript features, provides a nice developer experience, and optimizes your app for production. You'll need to have Node >= 14.0.0 and npm >= 5.6 on your machine so you have to install –Node and npm in your system.
- It automatically set up and creates a file system and folders you need to get started with react.

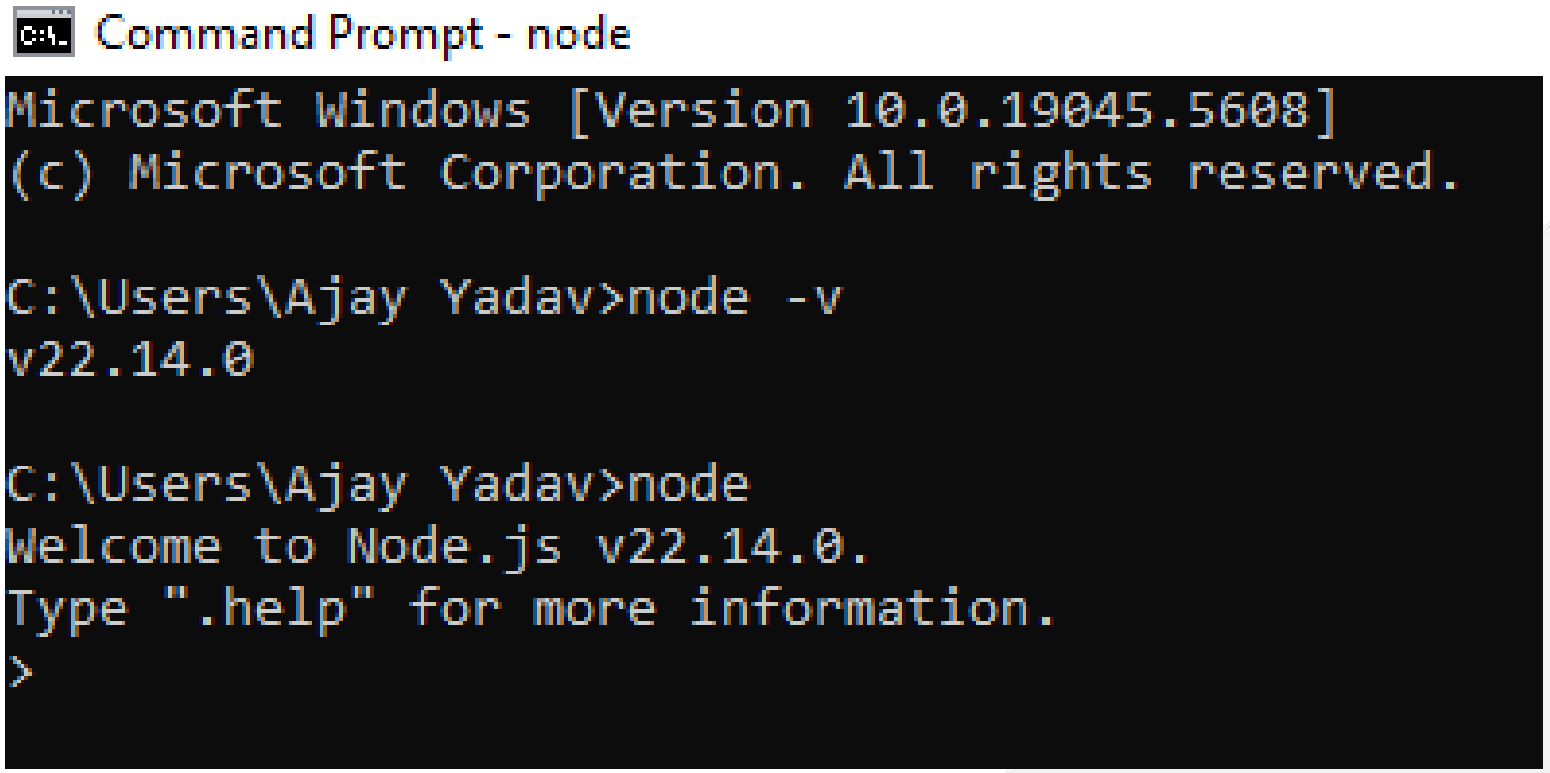
Installation of node js and npm(node package manager-

Steps-Go to -https://nodejs.org/en



To check node is installed or not

Go to terminal –



Now Create a New React App -Now create your first app in react by using these command in terminal

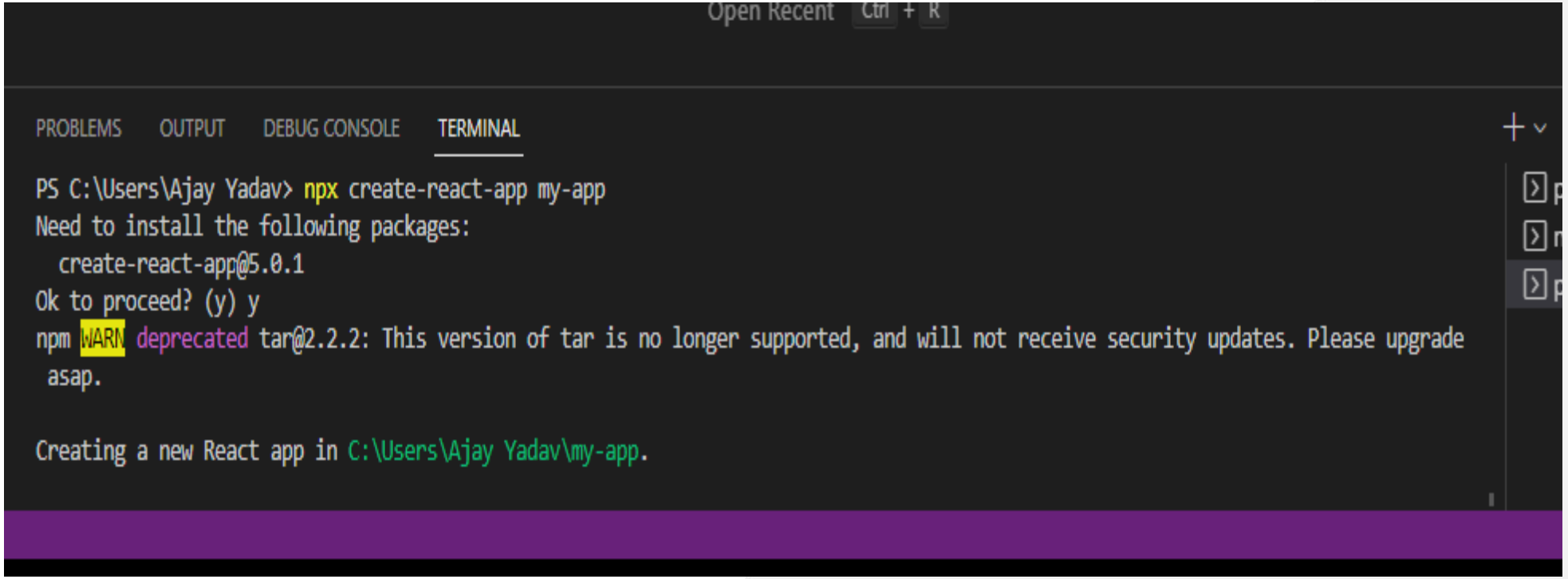
- **npx create-react-app my-app**
- **cd my-app**
- **npm start**

NPM: The npm stands for **Node Package Manager** and it is the default package manager for **Node.js.**

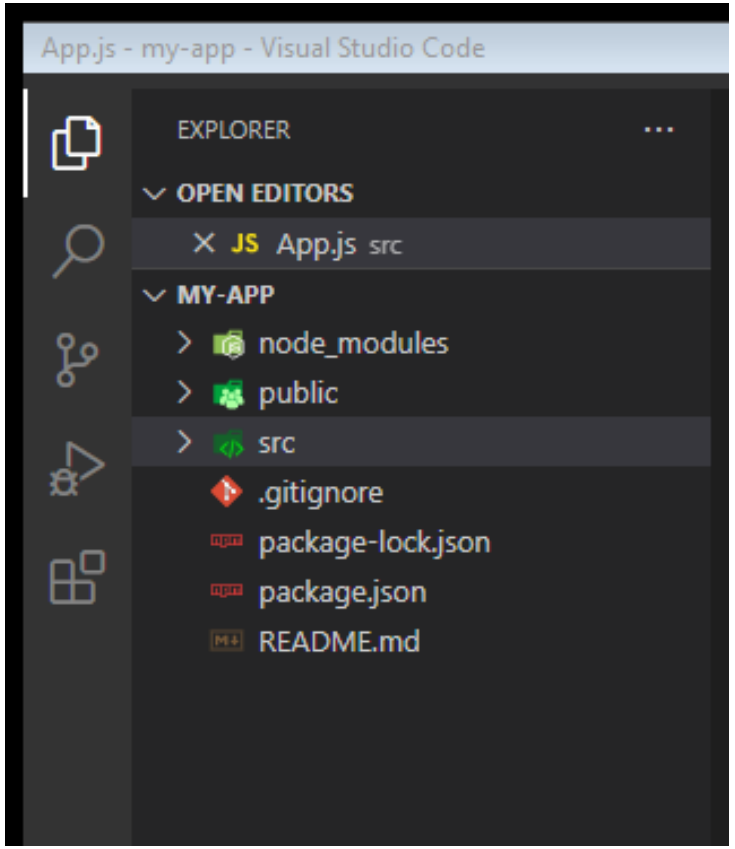
NPX: The npx stands for **Node Package Execute** and it comes with the npm, when you installed npm above 5.2.0 version then automatically npx will installed. It is an npm package runner that can execute any package that you want from the npm registry without even installing that package.

Command-
npx create-react-app my-app

Running any of these commands will create a directory called my-app inside the current folder. Inside that directory, it will generate the initial project structure and install the transitive dependencies:

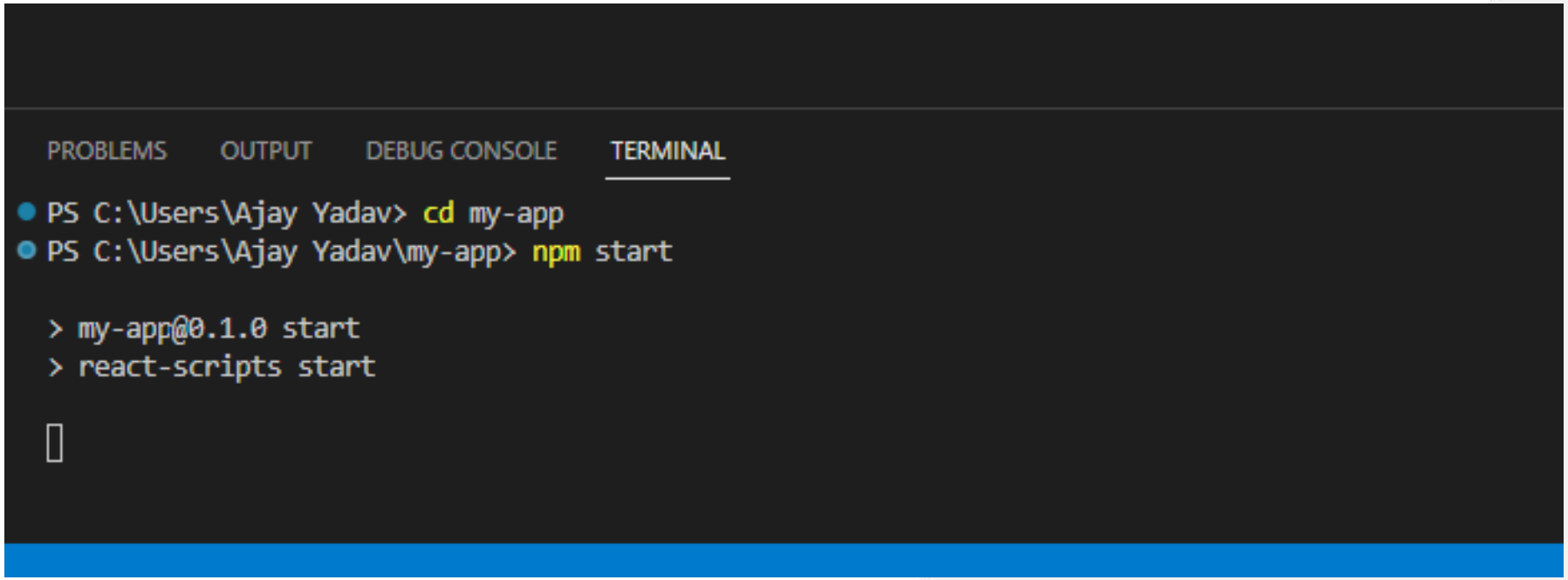


Folder and File Structure

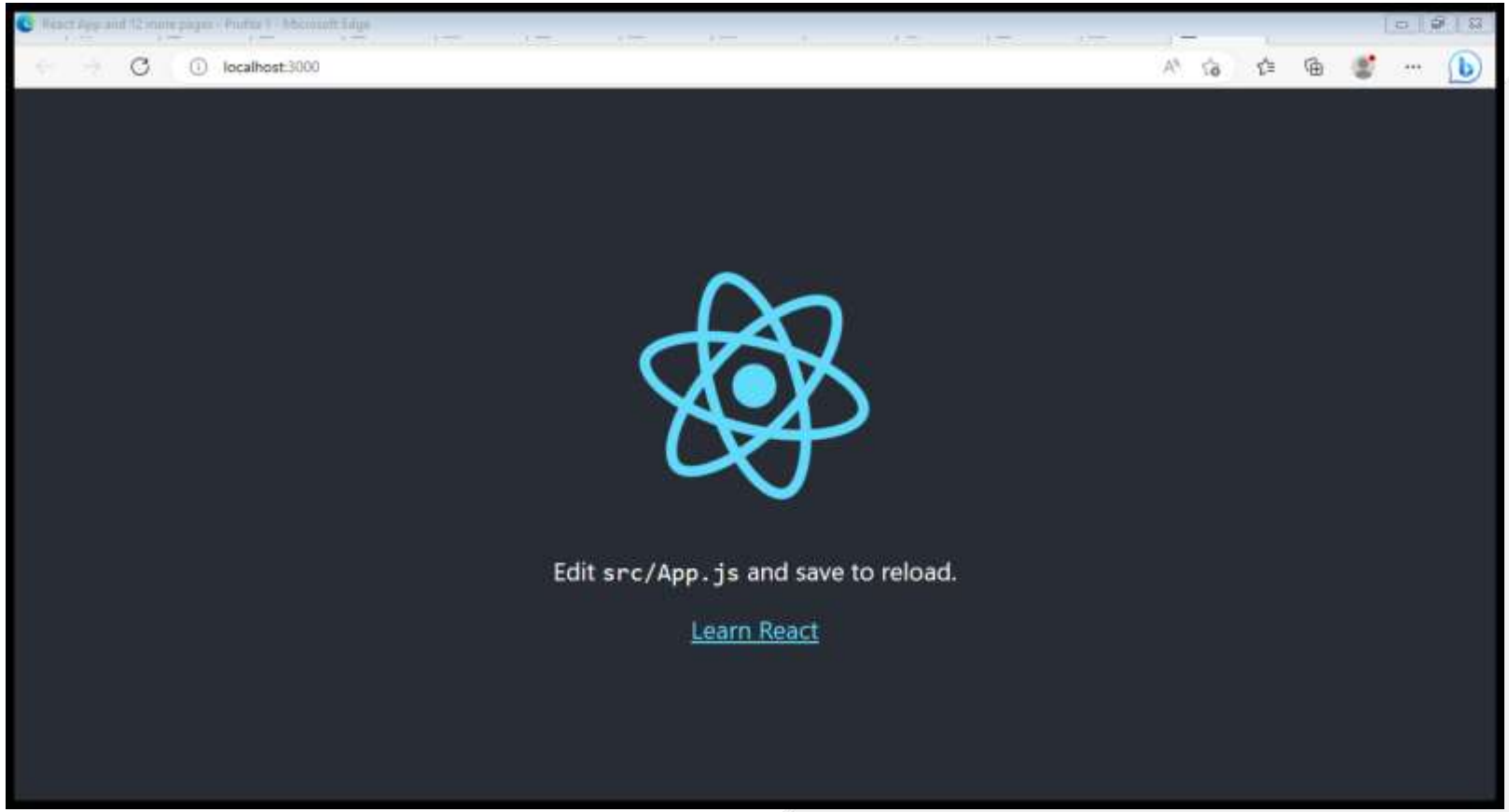



```
my-app
├── README.md
├── node_modules
├── package.json
├── .gitignore
├── public
│   ├── favicon.ico
│   ├── index.html
│   ├── logo192.png
│   ├── logo512.png
│   ├── manifest.json
│   └── robots.txt
└── src
    ├── App.css
    ├── App.js
    ├── App.test.js
    ├── index.css
    ├── index.js
    ├── logo.svg
    ├── serviceWorker.js
    └── setupTests.js
```

- cd my-app
- npm start



Your React project on browser-



CU
CHANDIGARH UNIVERSITY

JS App.js ×

src > JS App.js > App

```
1   import logo from './logo.svg';
2   import './App.css';
3
4   function App() {
5     return (
6       <div className="App">
7         <header className="App-header">
8           <img src={logo} className="App-logo" alt="logo" />
9           <p>
10            Edit <code>src/App.js</code> and save to reload.
11          </p>
12          <a
13            className="App-link"
14            href="https://reactjs.org"
15            target="_blank"
16            rel="noopener noreferrer"
17          >
18            Learn React
19          </a>
20        </header>
21      </div>
22    );
```

Example-

Replace all the content inside the with a <h1> element. See the changes in the browser when you click Save.

```
function App() {
  return (
    <div className="App">
      <h1>Hello World!</h1>
    </div>
  );
}

export default App;
```

2.CDN Links- go to reactjs.org

Both React and ReactDOM are available over a CDN.

```
<script crossoriginsrc="https://unpkg.com/react@18/umd/react.development.js"></script>
<script
crossoriginsrc="https://unpkg.com/reactdom@18/umd/reactdom.development.js"></script>
```

The versions above are only meant for development, and are not suitable for production. Minified and optimized production versions of React are available at:

```
<script crossorigin
src="https://unpkg.com/react@18/umd/react.production.min.js"></script> <script
crossorigin src="https://unpkg.com/react-dom@18/umd/react-
dom.production.min.js"></script>
```

Example-

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  </head>
  <body>

    <div id="mydiv"></div>

    <script type="text/babel">
      function Hello() {
        return <h1>Hello World!</h1>;
      }

      const container = document.getElementById('mydiv');
      const root = ReactDOM.createRoot(container);
      root.render(<Hello />)
    </script>

  </body>
</html>
```

- **Add React to an Existing Project**
- If you want to add some interactivity to your existing project, you don’t have to rewrite it in React. Add React to your existing stack, and render interactive React components anywhere.

- **You need to install Node.js for local development.** Although you can try React online or with a simple HTML page, realistically most JavaScript tooling you’ll want to use for development requires Node.js.

What is a Component?-

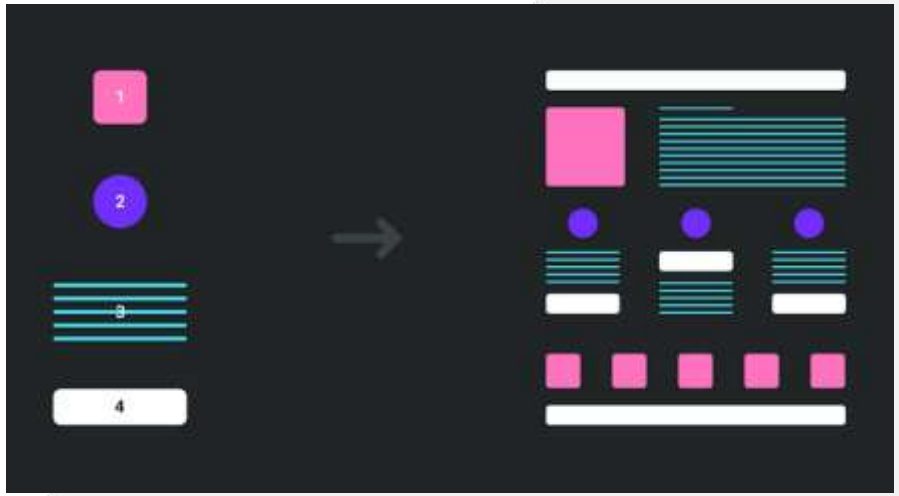
React Components are the building block of React Application. Components are independent and reusable bits of code. They serve the same purpose as JavaScript functions, but work in isolation and return HTML.

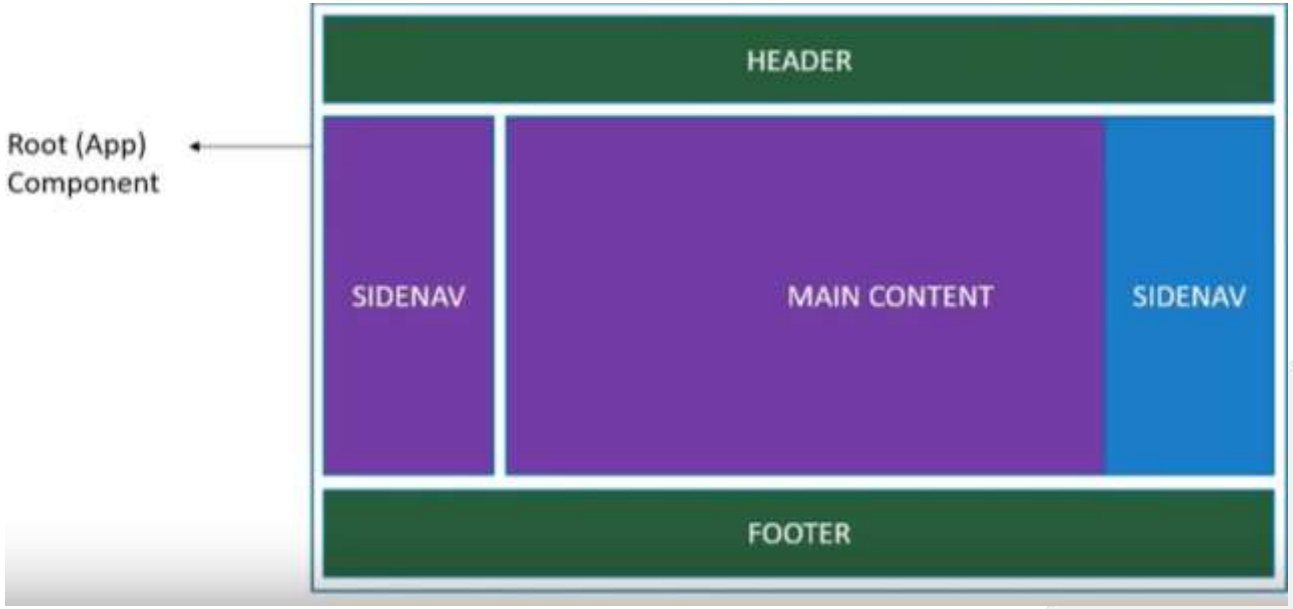
- **Components in React** serve as independent and reusable code blocks for UI elements.
- They represent different parts of a web page and contain both **structure (HTML)** and **behavior (JavaScript and library)**.
- They are similar to **JavaScript functions** and make creating and managing complex user interfaces easier by breaking them down into smaller, reusable pieces.
- They are the reusable code blocks containing logics and and UI elements
- A UI is broken down into multiple individual pieces called components. You can work on components independently and then merge them all into a **parent component** which will be your final UI.

Types of Components in React

In React, we mainly have two types of components:

1. **Function components and**
2. **Class components,**





1. Function Component

- Functional components are just like JavaScript functions that accept properties and return a React element.but Function components can be written using much less code.
- JavaScript functions that may or may not receive data as parameters. We can create a function that takes props(properties) as input and returns what should be rendered.
- The functional component is also known as a **stateless component** because they do not hold or manage state.

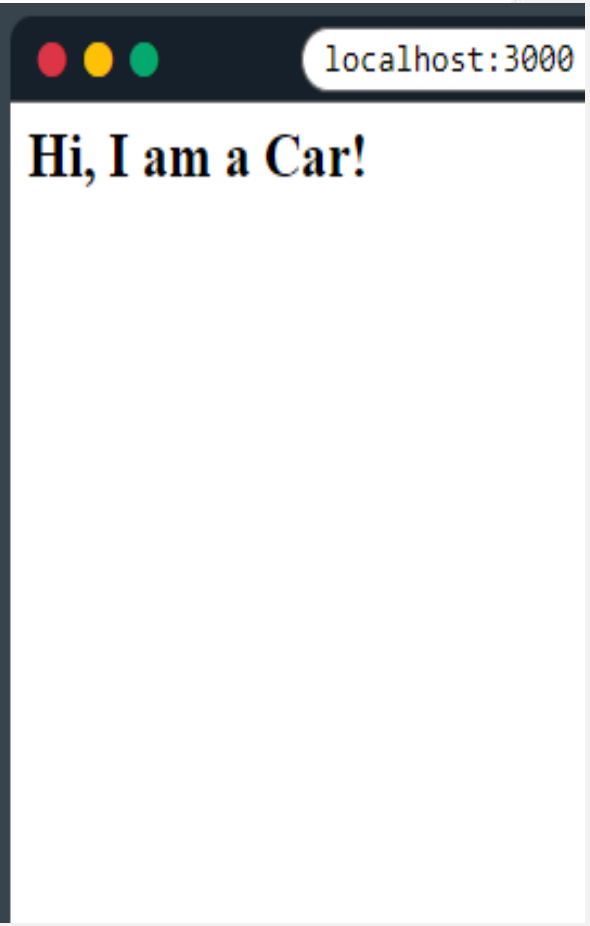
Syntax:

```
function demoComponent() {
    return (<h1>
                Welcome Message!
            </h1>);
}
```

Example

Display the Car component in the "root" element:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Car() {
  return <h2>Hi, I am a Car!</h2>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Car />);
```



2.Class Component- Class components are more complex than functional components.

- It requires to extend from **React. Component** and create a **render function** which **returns a React element (HTML Elements)**.
- You can pass data from one class to other class components.
- You can create a class by defining a class that extends Component and has a render function.
- The class component is also known as a **stateful component** because they can hold or manage local state.
- React components has a built-in state object.
- The state object is where you store property values that belong to the component.

Example

Create a Class component called `Car`

```
class Car extends React.Component {
  render() {
    return <h2>Hi, I am a Car!</h2>;
  }
}
```

Using the extends keyword, you can allow the current component to access all the component's properties, including the function, and trigger it from the child component.

In older React code bases, you may find Class components primarily used. It is now suggested to use Function components along with Hooks, which were added in React 16.8. There is an optional section on Class components for your reference. Hooks allow function components to have access to state and other React features.

React Rendering-

Rendering is the process of generating HTML markup to display web pages in the browser.

- "**Rendering**" is React calling your components. **On initial render, React will call the root component**. For subsequent renders, **React will call the function component** whose state update triggered the render.

- The **render()** method is then called to define the React component that should be rendered.

- Render in React JS is a fundamental part of class components.

render()-React renders HTML to the web page by using a function called render().

- The createRoot() function takes one argument, an HTML element. The purpose of the function is to define the HTML element where a React component should be displayed.

Example

Display a paragraph inside an element with the id of "root":

```
const container = document.getElementById('root');
const root = ReactDOM.createRoot(container);
root.render(<p>Hello</p>);
```

The result is displayed in the `<div id="root">` element:

```
<body>
  <div id="root"></div>
</body>
```

Rendering an Element into the DOM-

Let's say there is a somewhere in your HTML file:

To render a React element, first pass the DOM element to ReactDOM.createRoot(), then pass the React element to root.render():

```
const root = ReactDOM.createRoot(
  document.getElementById('root')
);
const element = <h1>Hello, world</h1>;
root.render(element);
```

Updating the Rendered Element

- React elements are immutable.
- Once you create an element, you can’t change its children or attributes.
- An element is like a single frame in a movie: it represents the UI at a certain point in time.

Consider this ticking clock example:

```
const root = ReactDOM.createRoot(
  document.getElementById('root')
);

function tick() {
  const element = (
    <div>
      <h1>Hello, world!</h1>
      <h2>It is {new Date().toLocaleTimeString()}.</h2>
    </div>
  );
  root.render(element);
}

setInterval(tick, 1000);
```

It calls root.render() every second from a setInterval() callback.

Example

Create a variable that contains HTML code and display it in the "root" node:

```
const myelement = (
  <table>
    <tr>
      <th>Name</th>
    </tr>
    <tr>
      <td>John</td>
    </tr>
    <tr>
      <td>Elsa</td>
    </tr>
  </table>
);

const container = document.getElementById('root');
const root = ReactDOM.createRoot(container);
root.render(myelement);
```

React Only Updates What's Necessary-

React DOM compares the element and its children to the previous one, and only applies the DOM updates necessary to bring the DOM to the desired state.

You can verify by inspecting the last example with the browser tools:

ECMAScript -was created to standardize JavaScript, and ES6 is the 6th version of ECMAScript, it was published in 2015, and is also known as ECMAScript 2015.

React uses ES6, and you should be familiar with some of the new features like:

- Classes
- Arrow Functions
- Variables (let, const, var)
- Array Methods like `.map()`
- Destructuring
- Modules
- Ternary Operator
- Spread Operator

Classes

ES6 introduced classes.
A class is a type of function, but instead of using the keyword `function` to initiate it, we use the keyword `class`,
and the properties are assigned inside a `constructor()` method.

Example

A simple class constructor:

```
class Car {
  constructor(name) {
    this.brand = name;
  }
}
```

React ES6 Arrow Functions

Arrow Functions

- Arrow functions allow us to write shorter function syntax:
- Normal-

```
hello = function() { return "Hello
World!"; }
```

- Arrow Function-

```
hello = () => { return
"Hello World!"; }
```

React ES6 Variables

Variables

Before ES6 there was only one way of defining your variables: with the `var` keyword. If you did not define them, they would be assigned to the global object.
Unless you were in strict mode, then you would get an error if your variables were undefined.
Now, with ES6, there are three ways of defining your variables: `var`, `let`, and `const`.

JavaScript Array map()

The map() Method

The `map()` method creates a new array with the results of calling a function for every array element.

Example

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(x => x * 2);
```

map() Parameters

The `map()` method takes three parameters:

- `currentValue` - The current element being processed
- `index` - The index of the current element (optional)
- `array` - The array that map was called upon (optional)

main.jsx | index.html

```jsx
import { createRoot } from 'react-dom/client'

const fruitlist = ['apple', 'banana', 'cherry'];

function App() {
  return (
    <ul>
      {fruitlist.map((fruit, index, array) => {
        return (
          <li key={fruit}>
            Name: {fruit}, Index: {index}, Array: {array}
          </li>
        );
      })}
    </ul>
  );
}

createRoot(document.getElementById('root')).render(
  <App />
)
```

localhost:5173

- Name: apple, Index: 0, Array: applebananacherry
- Name: banana, Index: 1, Array: applebananacherry
- Name: cherry, Index: 2, Array: applebananacherry

Destructuring-

is a feature in JavaScript (ES6) that allows you to **extract values from arrays or objects and assign them to variables in a clean way**.

```
const arr = [10, 20, 30];

const [a, b, c] = arr;

console.log(a); // 10
console.log(b); // 20
console.log(c); // 30
```

```html
<!DOCTYPE html>
<html>

<body>

<p id="demo"></p>

<script>
const vehicles = ['mustang', 'f-150',
'expedition'];

const [car, truck, suv] = vehicles;

//You can now access each variable separately:
document.getElementById('demo').innerHTML = truck;
</script>

</body>
</html>
```

f-150

React ES6 Spread Operator

Spread Operator

The JavaScript spread operator (`...`) allows us to quickly copy all or part of an existing array or object into another array or object.

```
<!DOCTYPE html>
<html>

<body>

<script>
const numbersOne = [1, 2, 3];
const numbersTwo = [4, 5, 6];
const numbersCombined = [...numbersOne, ...numbersTwo];

document.write(numbersCombined);
</script>

</body>
</html>
```

```
1,2,3,4,5,6
```

React ES6 Modules

Modules

JavaScript modules allow you to break up your code into separate files.
This makes it easier to maintain the code-base.
ES Modules rely on the `import` and `export` statements.

Export

You can export a function or variable from any file.
There are two types of exports: Named and Default.

Import

- You can import modules into a file in two ways, based on if they are named exports or default exports.
- Named exports must be destructured using curly braces. Default exports do not.

1. Named Export-

Used when you want to export **multiple values** from a file.

Example (Export File)

math.js

```
export const add = (a, b) => a + b;
export const sub = (a, b) => a - b;
```

Import in Another File

```
import { add, sub } from "./math";

console.log(add(5, 3)); // 8
console.log(sub(5, 3)); // 2
```

2.Default Exports

Let us create another file, named `message.js`, and use it for demonstrating default export.
You can only have one default export in a file.

Example

Insert the following code into the newly created file:

message.js

```
const message = () => {
  const name = "Tobias";
  const age = 18;
  return name + ' is ' + age + 'years old.';
};

export default message;
```

Example

Import named exports from the file person.js:

```
import { name, age } from "./person.js";
```

React ES6 Ternary Operator

Ternary Operator

The ternary operator is a simplified conditional operator like `if` / `else`.
Syntax: `condition ? <expression if true> : <expression if false>`
Here is an example using `if` / `else`

Example

Before:

```
if (authenticated) {
  renderApp();
} else {
  renderLogin();
}
```

Example

With Ternary

```
authenticated ? renderApp() : renderLogin();
```

React ES6 Template Strings

- Template Strings
- Template strings allow you to write strings that span multiple lines and include embedded expressions:

Example

Before:

```
const name = "John";
const age = 30;
const message = "Hello, " + name + "!\n" +
"You are " + age + " years old.";
```

Example

With Template Strings:

```
const name = "John";
const age = 30;
const message = `Hello, ${name}!
You are ${age} years old.`;
```

HTML element in React-

1.React.createElement ()-

React.createElement is a fundamental method of React JS. The main use of React.createElement is the Creation of a React component.

It is the JavaScript format for creating react components.

2.JSX- JSX allows us to write HTML in React

React.createElement()- React provides the createElement function to create and return elements.

React.createElement(
type,
[props],
[...children]
)

- Create and return a new React element of the given type. The type argument can be either a tag name string (such as 'div' or 'span'), a React component type (a class or a function), or a React fragment type.
- Code written with JSX will be converted to use React.createElement(). You will not typically invoke React.createElement() directly if you are using JSX.

React.createElement()

JS app.js

```
JS app.js > ...
1
2   const title= React.createElement(
3       'h1',
4       {id:'title-id',title:'this is title'},
5       'My first React Element !'
6   )
7
8   //<h1 id="title-id" title="this is tile">My first React Element !</h1>
```

Render an Element-

The 'react-dom' library provides DOM-specific methods. The one you'll use the most is ReactDOM.render(), which renders React elements to the DOM.

Rendering is considered as Mounting in React .

Syntax-

```
ReactDOM.render(
    title,
    document.getElementById('root')
);
```

What to render

Where to render

JSX-

JSX is an extension to the JavaScript language that uses a markup-like syntax to create React elements. Most React developers write their UI using JSX because it resembles writing HTML.

- **JSX stands for JavaScript XML(JavaScript Extensible Markup Language).**
- **JSX allows us to write HTML in React.**
- **JSX makes it easier to write and add HTML in React.**

Coding JSX

- JSX allows us to write HTML elements in JavaScript and place them in the DOM without any createElement() and/or appendChild() methods.
- JSX converts HTML tags into react elements.
- You are not required to use JSX, but JSX makes it easier to write React applications.

Example 1

JSX:

```
const myElement = <h1>I Love JSX!</h1>;

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```

Without JSX same code-

Example 2

Without JSX:

```
const myElement = React.createElement('h1', {}, 'I do not use JSX!');

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```

Expressions in JSX

With JSX you can write expressions inside curly braces { }.
The expression can be a React variable, or property, or any other valid JavaScript expression. JSX will execute the expression and return the result:

Example

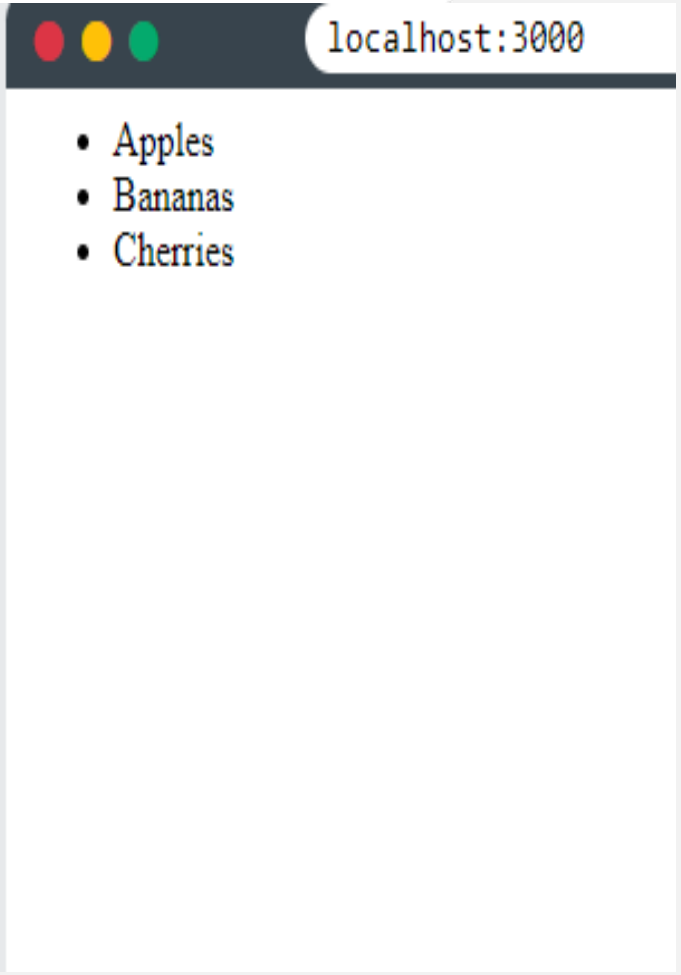
Execute the expression `5 + 5`:

```
const myElement = <h1>React is {5 + 5} times better with JSX</h1>;
```

Inserting a Large Block of HTML

To write HTML on multiple lines, put the HTML inside parentheses:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

const myElement = (
  <ul>
    <li>Apples</li>
    <li>Bananas</li>
    <li>Cherries</li>
  </ul>
);

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```

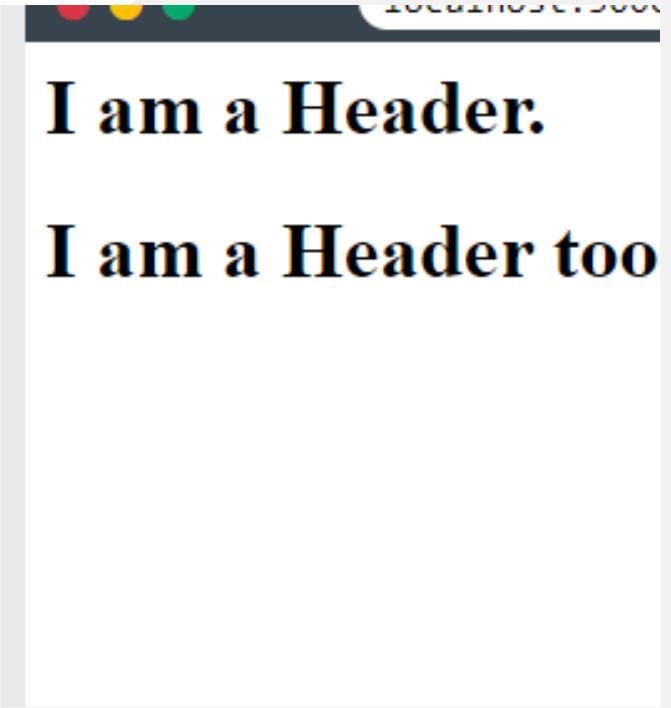


One Top Level Element Rule in React-

The HTML code must be wrapped in *ONE* top level element.
So if you like to write *two* paragraphs, you must put them inside a parent element, like a div element.

Note-JSX will throw an error if the HTML is not correct, or if the HTML misses a parent element.

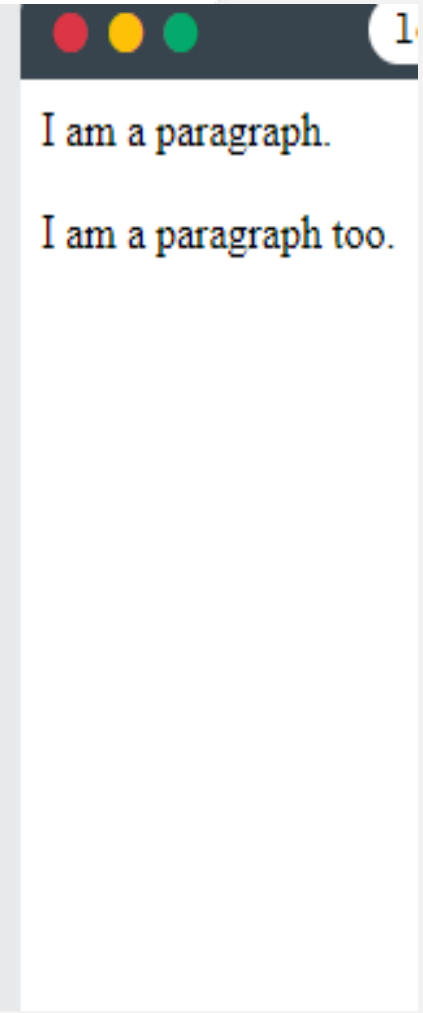
```
import React from 'react';
import ReactDOM from 'react-do/client';

const myElement = (
  <div>
    <h1>I am a Header.</h1>
    <h1>I am a Header too.</h1>
  </div>
);

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```



Alternatively, you can use a "**fragment**" to wrap multiple lines. This will prevent unnecessarily adding extra nodes to the DOM.

A fragment looks like an empty HTML tag: <></>.

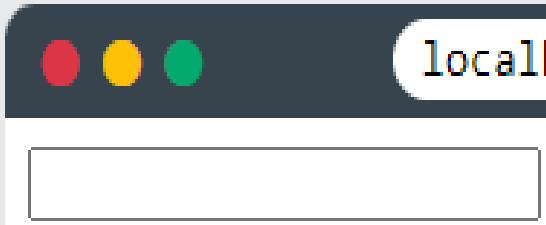
```
import React from 'react';
import ReactDOM from 'react-dom/client';

const myElement = (
    <>
      <p>I am a paragraph.</p>
      <p>I am a paragraph too.</p>
    </>
  );

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```



Elements Must be Closed

JSX follows XML rules, and therefore HTML elements must be properly closed. JSX will throw an error if the HTML is not properly closed.

```jsx
import React from 'react';
import ReactDOM from 'react-dom/client';

const myElement = <input type="text" />;

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```



Attribute class = className

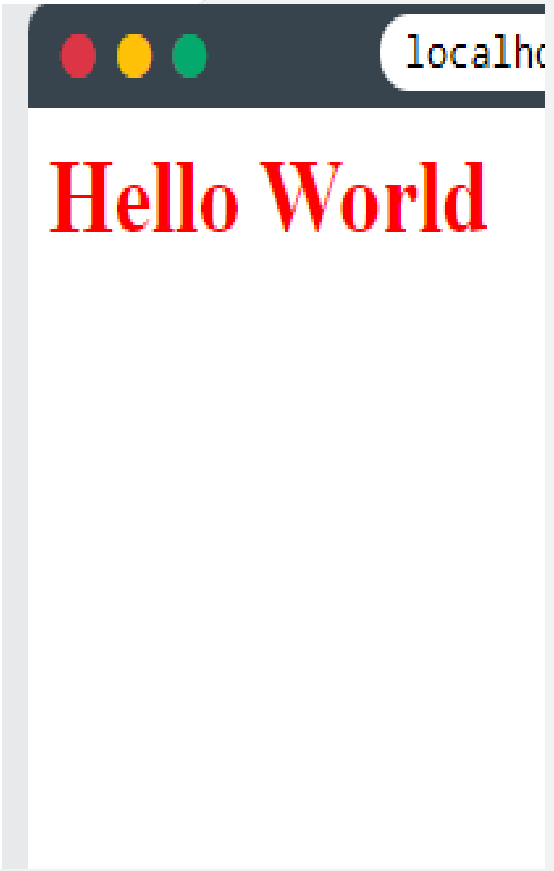
The class attribute is a much used attribute in HTML, but since JSX is rendered as JavaScript, and the class keyword is a reserved word in JavaScript, you are not allowed to use it in JSX.

- **JSX solved this by using className instead class. When JSX is rendered, it translates className attributes into class attributes.**
- **Use attribute className instead of class in JSX:**

Example-

Use attribute **className** instead of class in JSX:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

const myElement = <h1 className="myclass">Hello World</h1>;

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```



Conditions - if statements

React supports if statements, but not *inside* JSX.

To be able to use conditional statements in JSX, you should put the if statements outside of the JSX, or you could use a ternary expression instead:

- **Option 1:**
  Write if statements outside of the JSX code:
- **Option 2:**

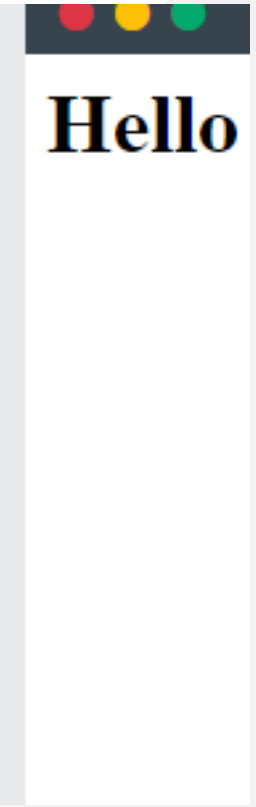
  Use ternary expressions instead: condition ? true : false

Conditional (ternary) operator-The **conditional (ternary) operator** is the only JavaScript operator that takes three operands: a condition followed by a question mark (?), then an expression to execute if the condition is truthy followed by a colon (:), and finally the expression to execute if the condition is falsy.

Example-Write "Hello" if x is less than 10, otherwise "Goodbye":

Write if statements outside of the JSX code:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

const x = 5;
let text = "Goodbye";
if (x < 10) {
  text = "Hello";
}

const myElement = <h1>{text}</h1>;

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```



Example-Write "Hello" if x is less than 10, otherwise "Goodbye":

Use ternary expressions instead:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

const x = 5;

const myElement = <h1>{(x) < 10 ? "Hello" : "Goodbye"}</h1>;

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```

React Events

Just like HTML DOM events, React can perform actions based on user events.
React has the same events as HTML: click, change, mouseover etc.

- **React events are written in camelCase syntax:**
- **onClick instead of onclick.**
- **React event handlers are written inside curly braces:**
- **onClick={shoot} instead of onClick="shoot()".**

React:

```
<button onClick={shoot}>Take the Shot!</button>
```

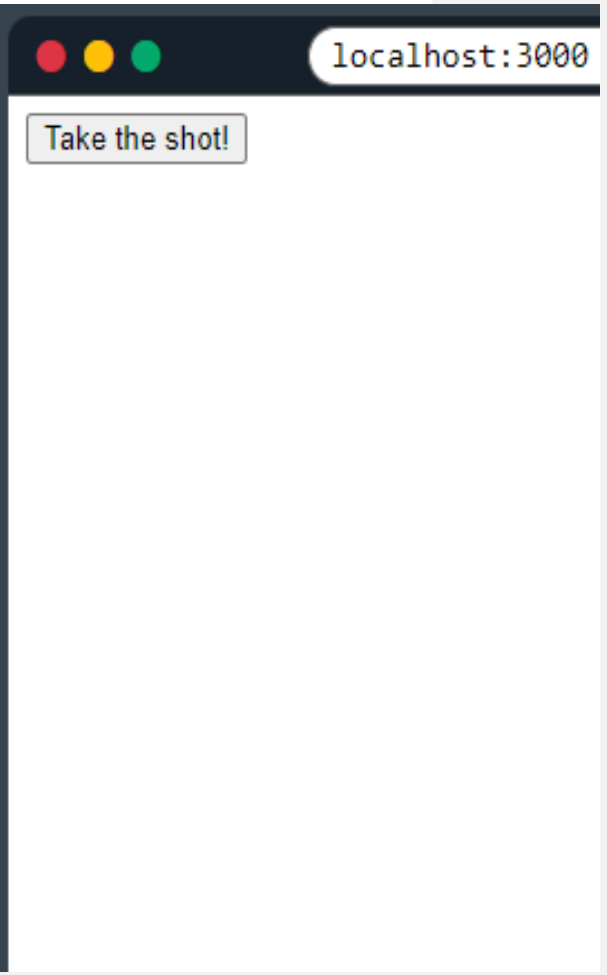
HTML:

```
<button onclick="shoot()">Take the Shot!</button>
```

Example:

Put the shoot function inside the Football component:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Football() {
  const shoot = () => {
    alert("Great Shot!");
  }

  return (
    <button onClick={shoot}>Take the shot!</button>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Football />);
```

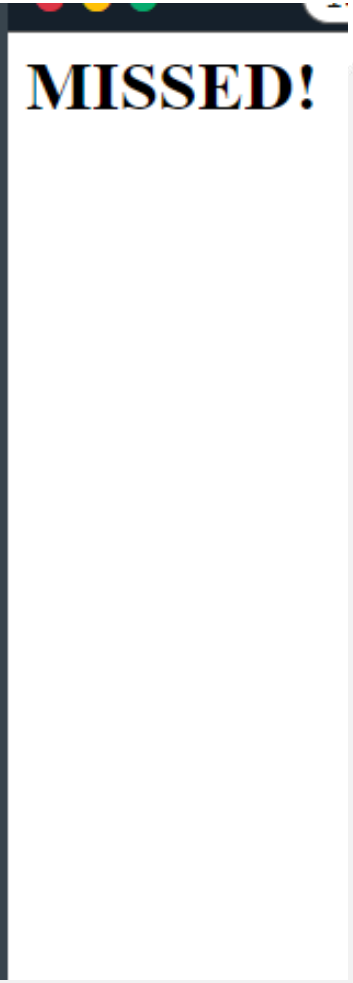


React Conditional Rendering

-if Statement

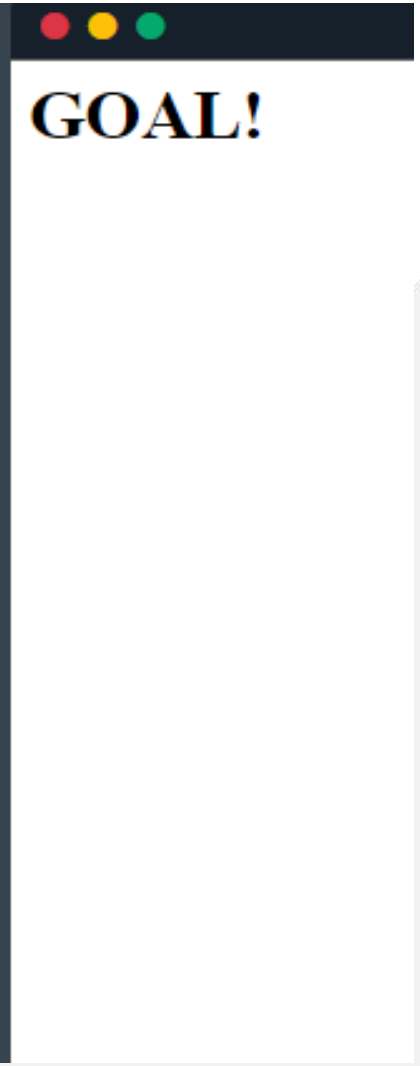
We can use the if JavaScript operator to decide which component to render.

```
function MissedGoal() {
  return <h1>MISSED!</h1>;
}

function MadeGoal() {
  return <h1>GOAL!</h1>;
}

function Goal(props) {
  const isGoal = props.isGoal;
  if (isGoal) {
    return <MadeGoal/>;
  }
  return <MissedGoal/>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Goal isGoal={false} />);
```



If condition is true

```jsx
import React from 'react';
import ReactDOM from 'react-dom/client';

function MissedGoal() {
  return <h1>MISSED!</h1>;
}

function MadeGoal() {
  return <h1>GOAL!</h1>;
}

function Goal(props) {
  const isGoal = props.isGoal;
  if (isGoal) {
    return <MadeGoal/>;
  }
  return <MissedGoal/>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Goal isGoal={true} />);
```

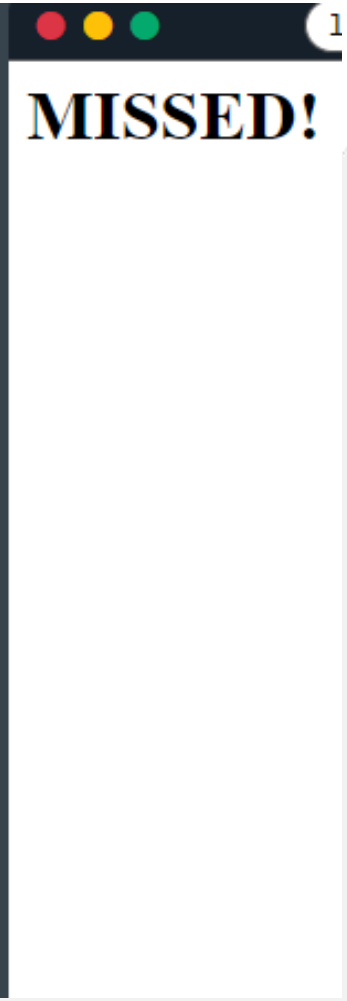


Ternary Operator

Another way to conditionally render elements is by using a ternary operator.

condition ? true : false

```
function MissedGoal() {
  return <h1>MISSED!</h1>;
}

function MadeGoal() {
  return <h1>GOAL!</h1>;
}

function Goal(props) {
  const isGoal = props.isGoal;
  return (
    <>
      { isGoal ? <MadeGoal/> : <MissedGoal/> }
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Goal isGoal={false} />);
```



Logical && Operator-

Another way to conditionally render a React component is by using the && operator.
If cars.length > 0 is equates to true, the expression after && will render.

```
function Garage(props) {
  const cars = props.cars;
  return (
    <>
      <h1>Garage</h1>
      {cars.length > 0 &&
        <h2>
          You have {cars.length} cars in your garage.
        </h2>
      }
    </>
  );
}

const cars = ['Ford', 'BMW', 'Audi'];
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Garage cars={cars} />);
```

Garage

You have 3 cars in your garage.

React Lifecycle (Lifecycle of Components)-

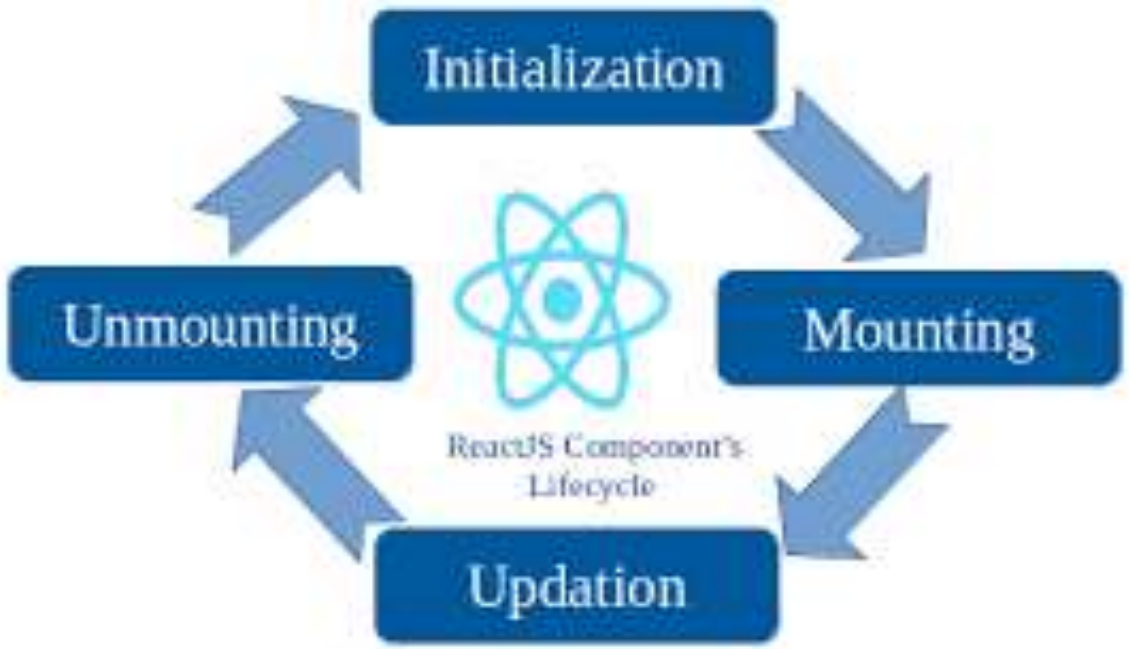
- **React Lifecycle** is defined as the series of methods that are invoked in different stages of the component's existence.

- **React lifecycle** starts from its initialization and ends when it is unmounted from the DOM.

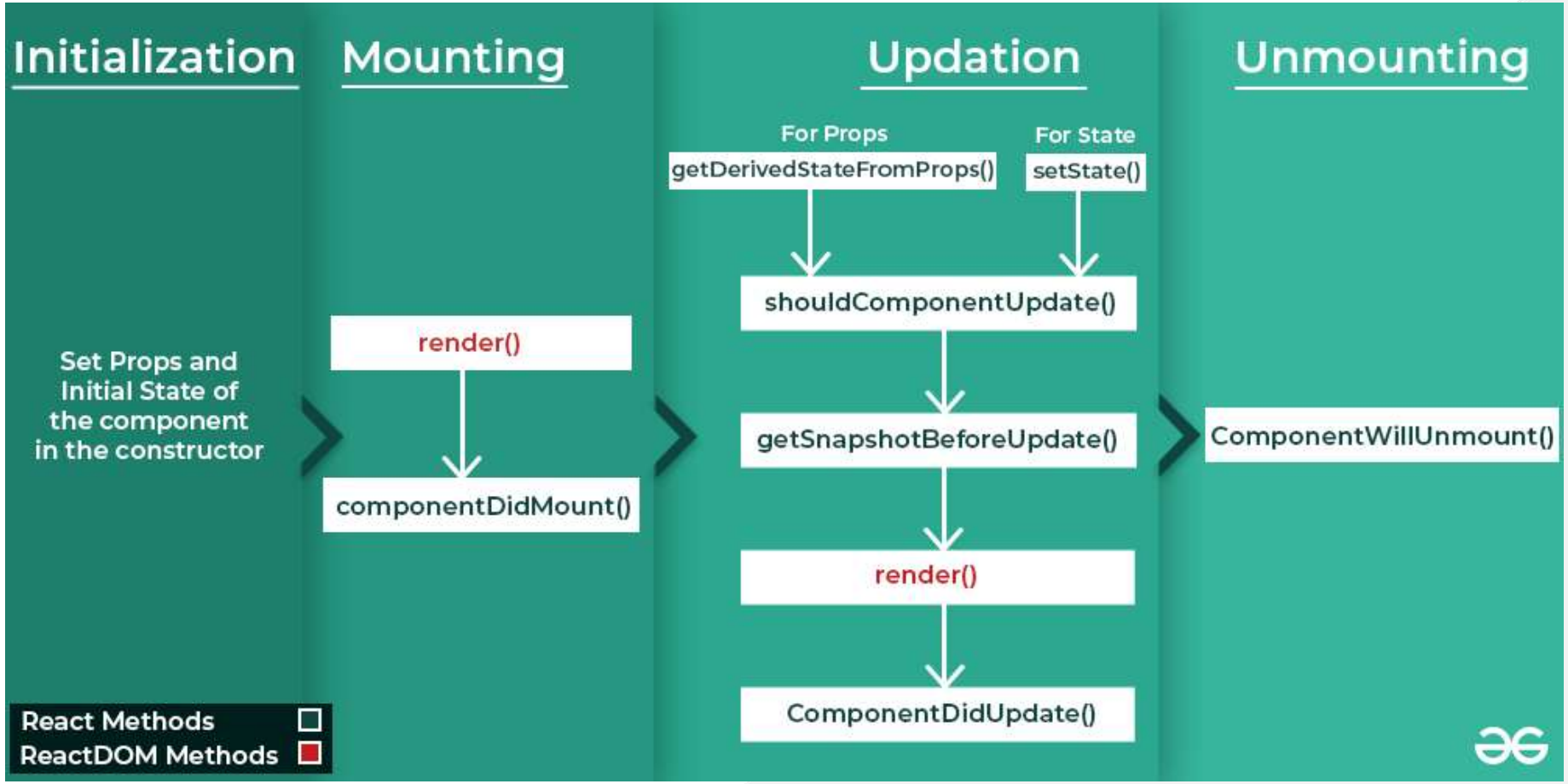
- There are several methods defined for different phases of the lifecycle of React Components.

A React Component can go through four stages of its life as follows.

1. **Initializing**
2. **Mounting**
3. **Updation**
4. **Unmounting**



Phases of Lifecycle in React Components-



1. Initialization-

- In this phase, the developer has to define the **props** and **initial state of the component** this is generally done in the constructor of the component.

Example-

```
class Clock extends React.Component {
constructor(props)
{
// Calling the constructor of
// Parent Class React.Component
super(props);

// Setting the initial state
this.state = { date : new Date() };
}
}
```

Note-The constructor is **a method used to initialize an object's state in a class**. It automatically called during the creation of an object in a class.

2. Mounting-

- Mounting means to be initialized and inserted in the DOM. Mounting is the initial phase in which the instance of the component is created and inserted into the DOM. When the component gets successfully inserted into the DOM, the component is said to be mounted..
- React follows a default procedure in the Naming Conventions of these predefined functions where the functions containing “**Will”** represents before some specific phase and “**Did**” represents after the completion of that phase.

The mounting phase consists of these predefined functions as described below.

1. **constructor**
2. **static getDerivedStateProps**
3. **render()**
4. **componentDidMount()**

1.Constructor- The constructor is **a method used to initialize an object's state in a class**. It automatically called during the creation of an object in a class.

2.static getDerivedStateProps-(To update the state)

- This method is called **before the rendering** or before any updation of the component. **This method update the state, before the rendering of the component.** This method is called on every rerendering of the component.

Syntax

static getDerivedStateFromProps(props, state)

- **props** – Props passed to component
- **state** – Previous state of component

3.render() method-

Responsible for rendering JSX and updating the DOM.

4.componentDidMount() Function

This function is invoked right after the component is mounted on the DOM i.e. this function gets invoked once after the render() function is executed for the first time

Initialization of constructor-

```
import React from 'react';
import ReactDOM from 'react-dom/client';

class Header extends React.Component {
  constructor(props) {
    super(props);
    this.state = {favoritecolor: "red"};
  }
```

static getDerivedStateProps-(To update the state)

```
import React from 'react';
import ReactDOM from 'react-dom/client';

class Header extends React.Component {
  constructor(props) {
    super(props);
    this.state = {favoritecolor: "red"};
  }
  static getDerivedStateFromProps(props, state) {
    return {favoritecolor: props.favcol };
  }
```

Rendering-

```
  }
  render() {
    return (
      <h1>My Favorite Color is {this.state.favoritecolor}</h1>
    );
  }
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Header favcol="yellow"/>);
```

componentDidMount()

- If you define the componentDidMount method, React will call it when your component is added *(mounted)* to the screen.
- This is a common place to start data fetching, set up subscriptions, or manipulate the DOM nodes.

```jsx
import React from 'react';
import ReactDOM from 'react-dom/client';

class Header extends React.Component {
  constructor(props) {
    super(props);
    this.state = {favoritecolor: "red"};
  }
  componentDidMount() {
    setTimeout(() => {
      this.setState({favoritecolor: "yellow"})
    }, 1000)
  }
  render() {
    return (
      <h1>My Favorite Color is {this.state.favoritecolor}</h1>
    );
  }
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Header />);
```

3.Updating-

The next phase in the lifecycle is when a component is *updated*. A component is updated whenever there is a change in the component's state or props.

Updation is the phase where the states and props of a component are updated followed by some user events such as clicking, pressing a key on the keyboard, etc.

React has five built-in methods that gets called, in this order, when a component is updated:

1. getDerivedStateFromProps()
2. shouldComponentUpdate()
3. render()
4. getSnapshotBeforeUpdate()
5. componentDidUpdate()

4. Unmounting-

- **The next phase in the lifecycle is when a component is removed from the DOM, or unmounting as React likes to call it.**

- **This is the final phase of the lifecycle of the component** which is the phase of unmounting the component from the DOM.

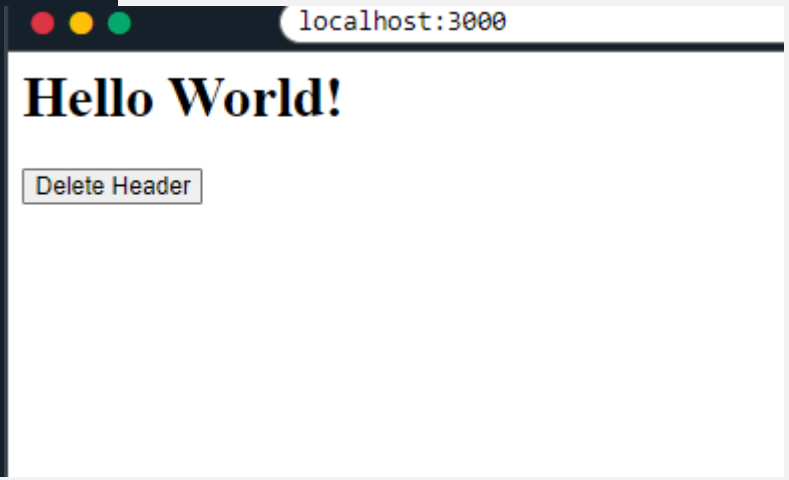
React has only one built-in method that gets called when a component is unmounted:

1.componentWillUnmount()

- **componentWillUnmount() Function:**
- This function is invoked before the component is finally unmounted from the DOM i.e. this function gets invoked once before the component is removed from the page and this denotes the end of the lifecycle

```
import React from 'react';
import ReactDOM from 'react-dom/client';

class Container extends React.Component {
  constructor(props) {
    super(props);
    this.state = {show: true};
  }
  delHeader = () => {
    this.setState({show: false});
  }
  render() {
    let myheader;
    if (this.state.show) {
      myheader = <Child />;
    };
    return (
      <div>
      {myheader}
      <button type="button" onClick={this.delHeader}>Delete Header</button>
      </div>
    );
```

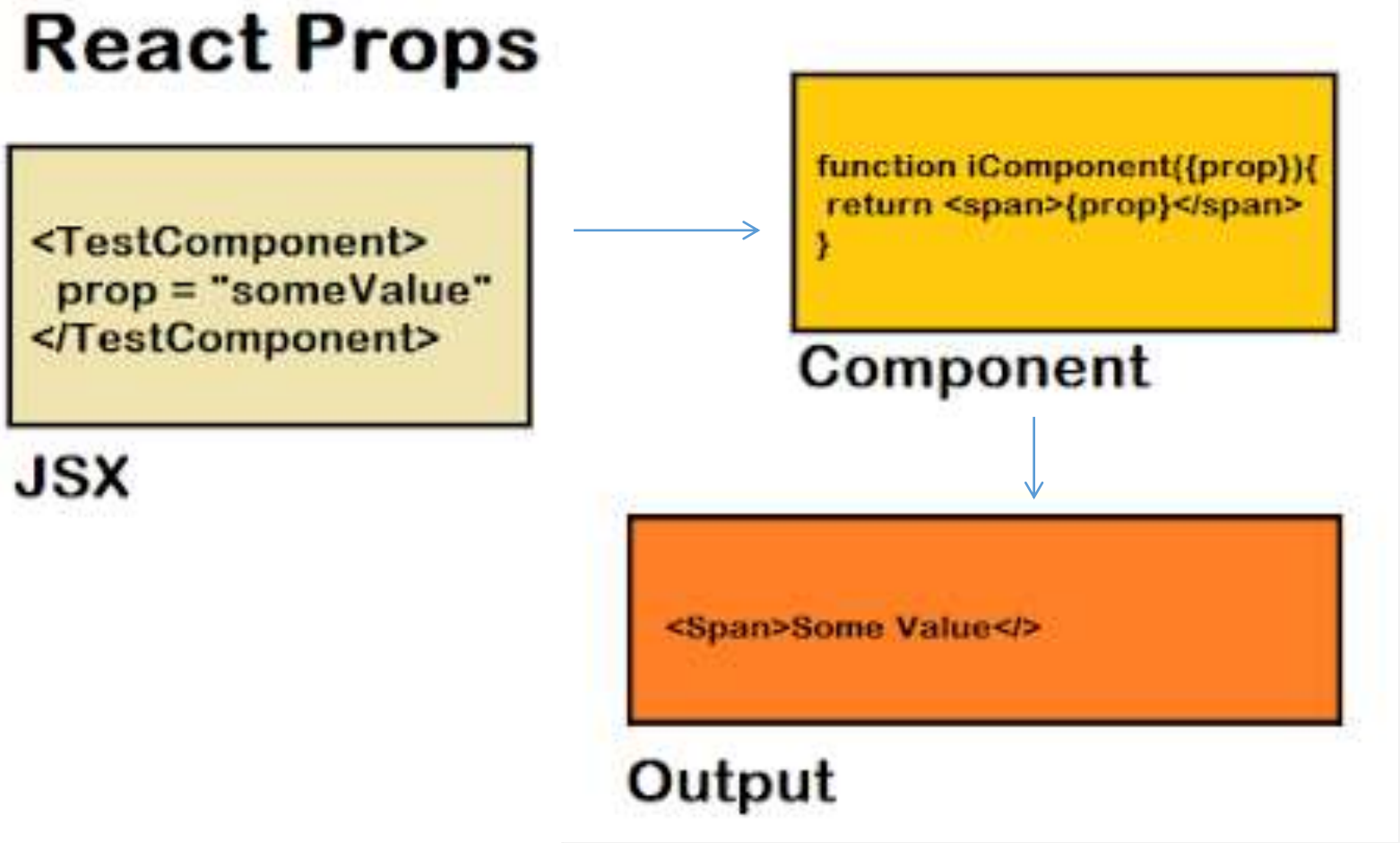
```
class Child extends React.Component {
  componentWillUnmount() {
    alert("The component named Header is about to be unmounted.");
  }
  render() {
    return (
      <h1>Hello World!</h1>
    );
  }
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Container />);
```



Props(Properties)-

- **Props is short for properties** and they are used to **pass data between React components. Props** in React are inputs that you pass into components.
- React components use ***props*** to communicate with each other. Every parent component can **pass some information to its child components by giving them props.**
- Props might remind you of HTML attributes, but you can pass any JavaScript value through them, **including objects, arrays, and functions.**
- **To send props into a component, use the same syntax as HTML attributes:**

- **React Props are like function arguments in JavaScript *and* attributes in HTML.**



Example-

To send props into a component, use the same syntax as HTML attributes:

Example

Add a "brand" attribute to the Car element:

```
const myElement = <Car brand="Ford" />;
```

The component receives the argument as a props object:

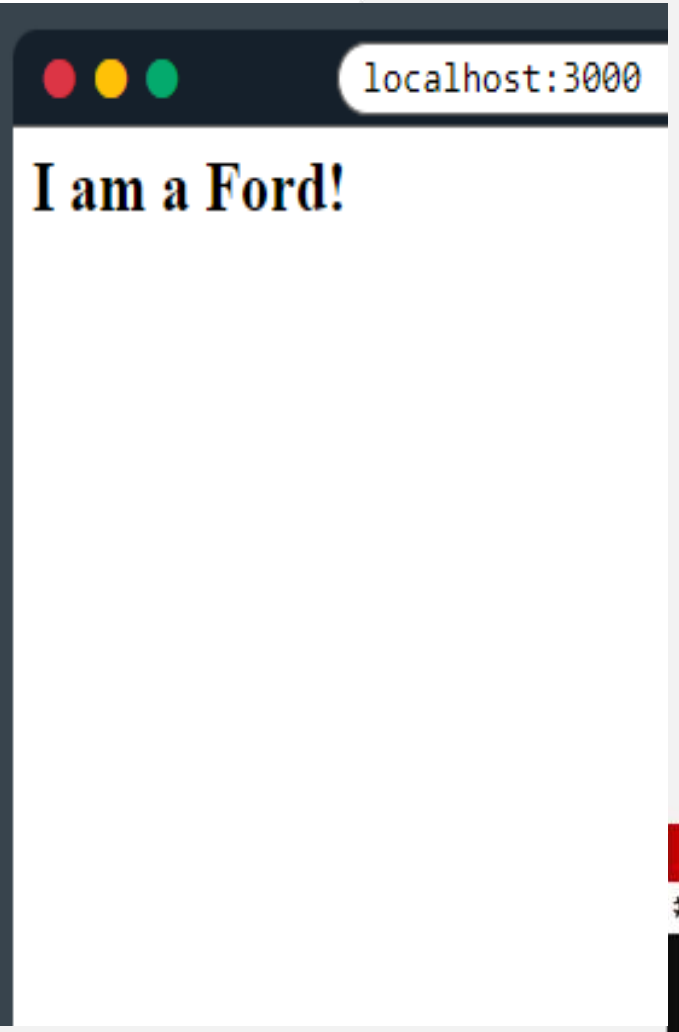
Example

Use the brand attribute in the component:

```
function Car(props) {
  return <h2>I am a { props.brand }!</h2>;
}
```

Here is an example of how data can be passed by using props:

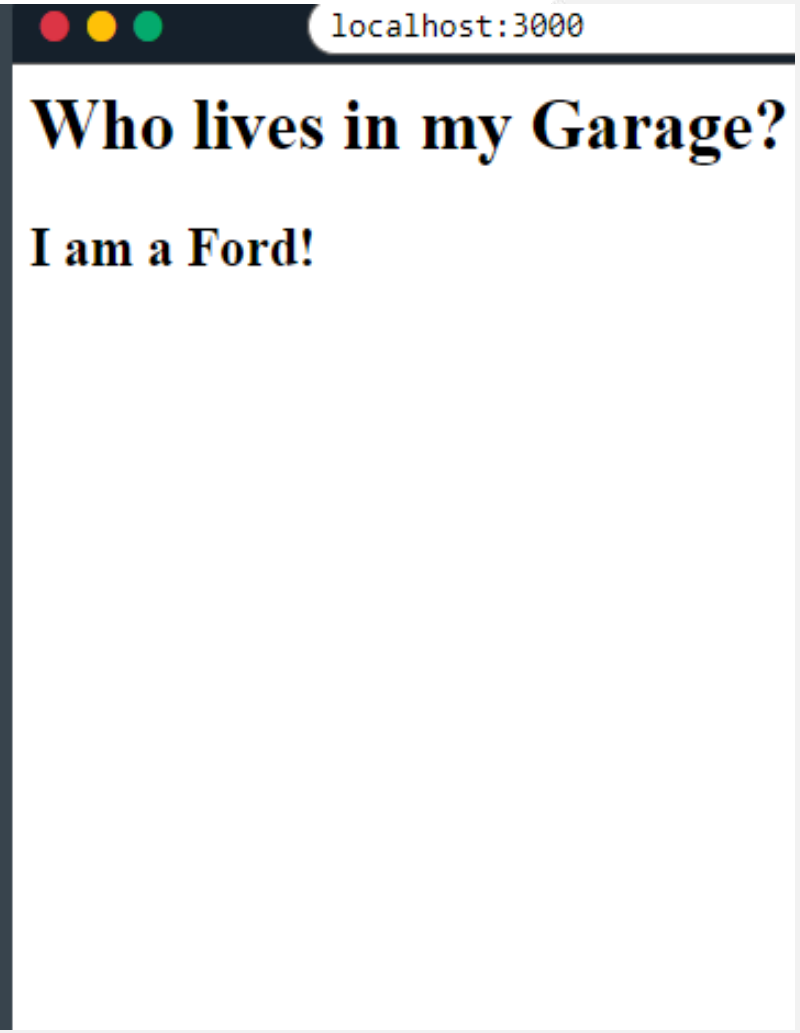
```jsx
import React from 'react';
import ReactDOM from 'react-dom/client';

function Car(props) {
  return <h2>I am a { props.brand }!</h2>;
}

const myElement = <Car brand="Ford" />;

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```



Pass Data-

Props can be use to pass data from one component to another, as parameters.

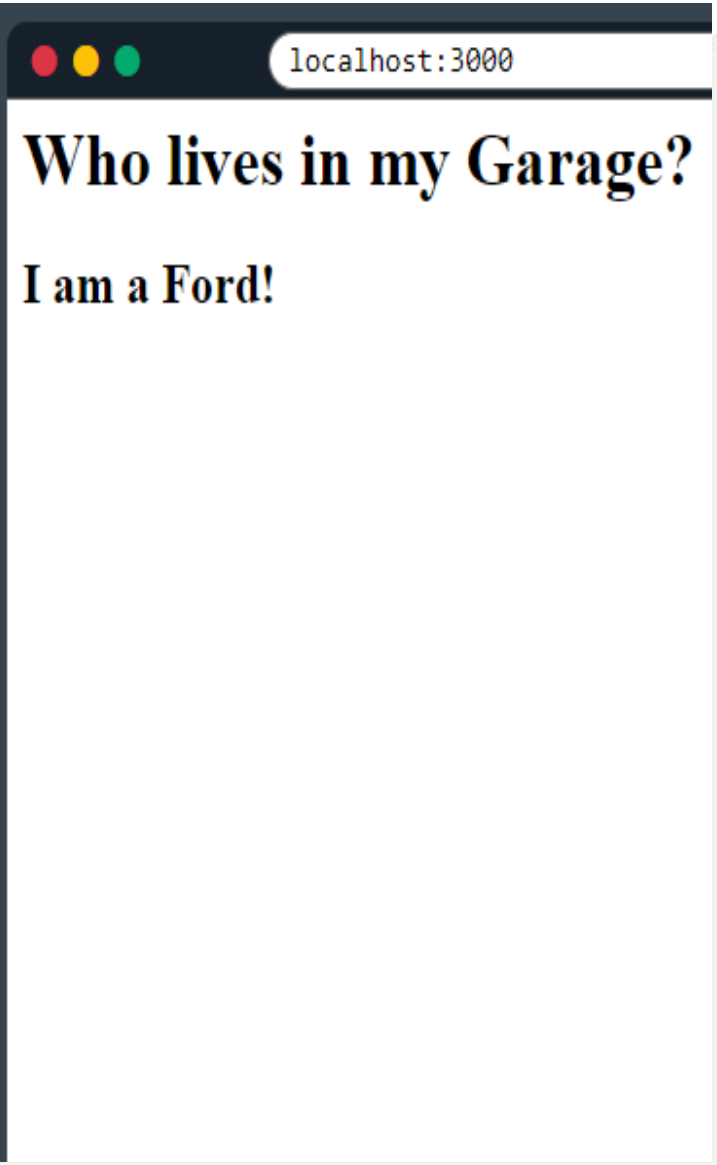
```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Car(props) {
  return <h2>I am a { props.brand }!</h2>;
}

function Garage() {
  return (
    <>
      <h1>Who lives in my garage?</h1>
      <Car brand="Ford" />
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Garage />);
```



Example

Create a variable named carName and send it to the Car component:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Car(props) {
  return <h2>I am a { props.brand }!</h2>;
}

function Garage() {
  const carName = "Ford";
  return (
    <>
      <h1>Who lives in my garage?</h1>
      <Car brand={ carName } />
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Garage />);
```



React Forms

Just like in HTML, React uses forms to allow users to interact with the web page. You add a form with React like any other element:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function MyForm() {
  return (
    <form>
      <label>Enter your name:
        <input type="text" />
      </label>
    </form>
  )
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<MyForm />);
```

Handling Forms

- Handling forms is about how you handle the data when Form changes value or gets submitted.
- In HTML, form data is usually handled by the DOM.
- In React, form data is usually handled by the components.
- When the data is handled by the components, all the data is stored in the component state.
- You can control changes by adding event handlers in the onChange attribute.

React State- State is a built-in object used to store data that can change over time in a component.

When state changes → UI automatically updates (re-render)

State = Dynamic data of a component

- React components has a built-in state object.
  The state object is where you store property values that belong to the component.
  When the state object changes, the component re-renders.
- React state management is a process for managing the data that React components need in order to render themselves. This data is typically stored in the component's state object. When the state object changes, the component will re-render itself.
- setState() schedules an update to a component's state object. When state changes, the component responds by re-rendering.
- state → current value
- setState → function to update value
- useState() → React hook

State

A state is an object that **stores the values** of properties belonging to a component that could change over a period of time

A state can be modified based on the user action or network changes

The state object can store **multiple properties**

Every time the state of an object changes, React re-renders the component to the browser

`this.setState()` is used to change the value of the state object

The state object is initialized in the constructor

`setState()` function performs a shallow merge between the new and the previous state

React Hooks -Hooks are special functions that allow you to use React features (like Class Components could use: state ,lifecycle methods) inside functional components.

- Hooks allow function components to have access to state and other React features. Because of this, class components are generally no longer needed.
- Hooks allow us to "hook" into React features such as state and lifecycle methods.

Hooks = bridge between functional components and React features

Hooks allow you to:

- **store data (state)**
- **run logic after render (effects)**
- **share data globally (context)**

Hook Rules- There are 3 rules for hooks:

1. Hooks can only be called inside React function components.
2. Hooks can only be called at the top level of a component.
3. Hooks cannot be conditional.

You must import Hooks from react at the top of component.

Example-like this- import { useState } from "react";

Different hooks and their purpose-

Hook Name	Usage / When to Use	Syntax Example	Purpose (What it does)
useState	When you need to store changing data	const [count, setCount] = useState(0);	Manages state (data that changes)
useEffect	When you need side effects (API, timer)	useEffect(() => { }, [deps]);	Runs code after render
useContext	When you need global data (no props drilling)	const value = useContext(MyContext);	Access shared/global data
useRef	When accessing DOM or storing value	const ref = useRef();	Stores value without re-render
useReducer	When state logic is complex	const [state, dispatch] = useReducer(reducer, init);	Manages complex state logic

Different hooks and their purpose-

Hook Name	Usage / When to Use	Syntax Example	Purpose (What it does)
useMemo	When optimizing heavy calculations	const value = useMemo(() => fn, [deps]);	Memoizes value (performance)
useCallback	When optimizing functions	const fn = useCallback(() => {}, [deps]);	Memoizes function
useLayoutEffect	When DOM update needed before paint	useLayoutEffect(() => {}, []);	Runs before browser paint
useImperativeHandle	With forwardRef customization	useImperativeHandle(ref, () => ({}));	Customize ref behavior
useId	When generating unique IDs	const id = useId();	Generates unique ID

1. useState (State Hook)

The React useState Hook allows us to track state in a function component.

Purpose-Store and update **dynamic data**

How it work-

Syntax-const [count, setCount] = useState(0);

- useState(0) → initial value
- count → current value
- setCount() → update state

UI updates automatically

```jsx
import { useState } from 'react';
import { createRoot } from 'react-dom/client';

function FavoriteColor() {
  const [color, setColor] = useState("red");

  return (
    <>
      <h1>My favorite color is {color}!</h1>
      <button
        type="button"
        onClick={() => setColor("blue")}
      >Blue</button>
      <button
        type="button"
        onClick={() => setColor("red")}
      >Red</button>
      <button
        type="button"
        onClick={() => setColor("pink")}
      >Pink</button>
      <button
        type="button"
        onClick={() => setColor("green")}
      >Green</button>
```

My favorite color is green!

Blue Red Pink Green

Example- Here we are using the useState Hook to keep track of the application state. State generally refers to application data or properties that need to be tracked.

```jsx
function MyForm() {
  const [name, setName] = useState("");

  return (
    <form>
      <label>Enter your name:
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
      </label>
    </form>
  )
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<MyForm />);
```

Problem-Form Validation Using Single State Object

1.Design a form with fields like name and email, managed using one state object. Implement validation rules and display error messages dynamically in at least 3 field.

- 2. **Dynamic To-Do List (Add & Delete)**

Develop a To-Do list where users can add tasks and delete them individually. The application should prevent empty inputs and maintain a dynamic list using state

useEffect Hooks-

useEffect = Perform extra work after rendering (like API calls, timers, DOM updates)

The useEffect Hook allows you to perform side effects in your components.

Some examples of side effects are:

- API call
- Timer (setTimeout, setInterval)
- Event listeners
- DOM manipulation
- Logging

-

useEffect(<function>, <dependency>)

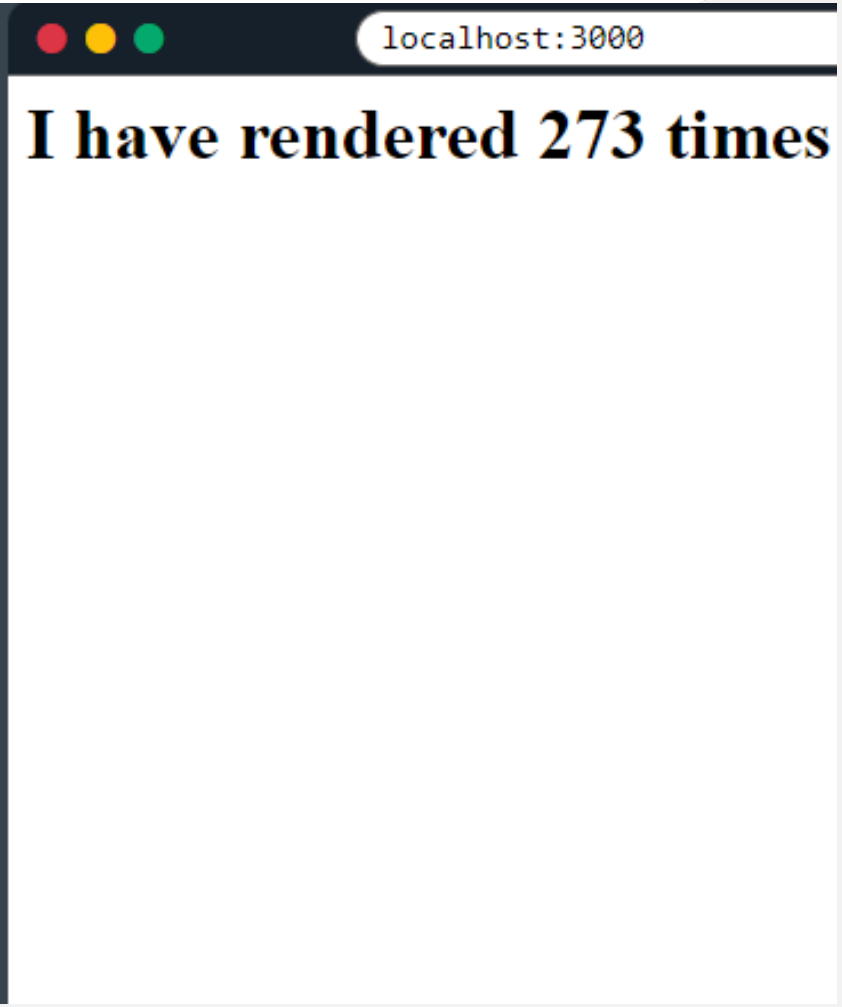
- useEffect runs on every render. That means that when the count changes, a render happens, which then triggers another effect.

Syntax-

```
useEffect(() => {
// code (side effect)
}, [dependencies]); //Controls when it runs
```

Example: Use setTimeout() to count 1 second after initial render:

```jsx
import { useState, useEffect } from "react";
import ReactDOM from "react-dom/client";

function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setTimeout(() => {
      setCount((count) => count + 1);
    }, 1000);
  });

  return <h1>I have rendered {count} times!</h1>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Timer />);
```



Problem-

1. Page Title Updater

Create a component where the browser tab title updates dynamically whenever a counter value changes.

2. Timer / Auto Counter

Develop a timer that automatically increments every second. The timer should stop when it reaches a specific limit.

3. Create an input field where every change is logged in the console using useEffect.

useContext Hook-

useContext = way to share data globally in React

- useContext is a **React Hook** that allows you to **use global data** (called context) in any component **without passing props manually** at every level.

The Problem

- State should be held by the highest parent component in the stack that requires access to the state.
- To illustrate, we have many nested components. The component at the top and bottom of the stack need access to the state.
- To do this without Context, we will need to pass the state as "props" through each nested component. **This is called "prop drilling**".

Problem-

1. Theme Context (Light/Dark Mode)

Create a global theme system using useContext where multiple components (Header, Content, Footer) can access and change the theme (Light/Dark).

2. User Authentication Context

- Develop an authentication system where user details (name, login status) are stored in a context. Display user information in different components without passing props manually.

3. Language Selector (Multi-language UI)

- Create a language context that allows switching between languages (e.g., English/Hindi). All components should update their text automatically when the language changes.

useRef Hook-

Access DOM elements directly
Store values that do NOT cause re-render

- The useRef Hook allows you to persist values between renders.
- It can be used to store a mutable value that does not cause a re-render when updated.
- It can be used to access a DOM element directly.

Does Not Cause Re-renders

- If we tried to count how many times our application renders using the useState Hook, we would be caught in an infinite loop since this Hook itself causes a re-render.
- To avoid this, we can use the useRef Hook.

Problem-

1. Auto Focus Input Field

Create an input box that automatically gets focus when the page loads using useRef.

2. Stop Watch (Without State for Timer Count)

Create a stopwatch where time is stored using useRef and updated using buttons (Start/Stop).

3. Scroll to Section

Create multiple sections and a button that scrolls to a specific section using useRef.

4. Count Button Clicks Without Re-render

- Create a button that counts how many times it is clicked but does NOT update UI, only logs in console.

Redux Js

Redux is a state management library used to manage and centralize application data. It is commonly used with React, but can also work with other frameworks.

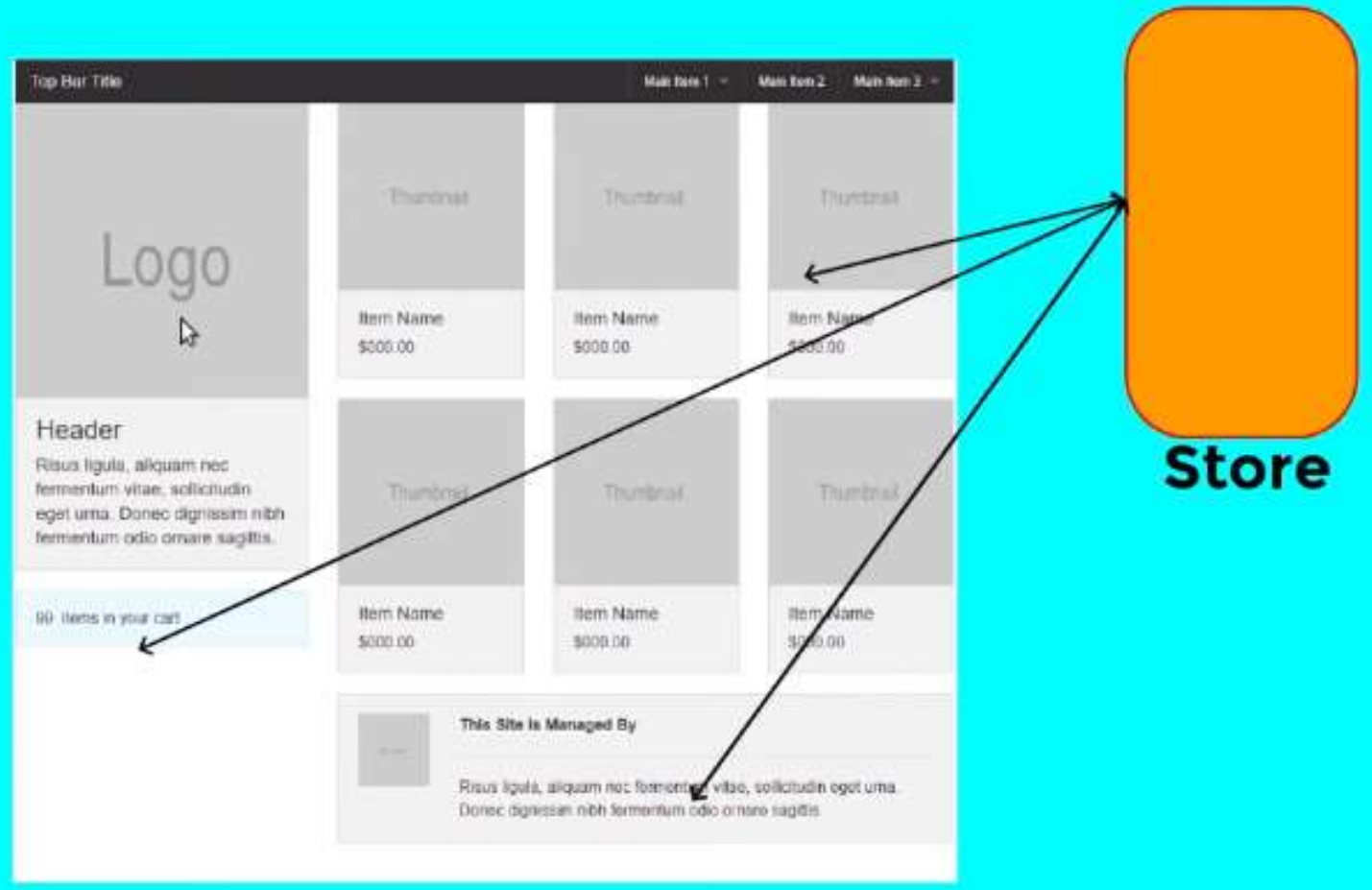
Redux introduction-

- Redux is the most powerful **state management tool** in the React. Js library, and it's frequently used by Frontend Engineers to build complex applications.

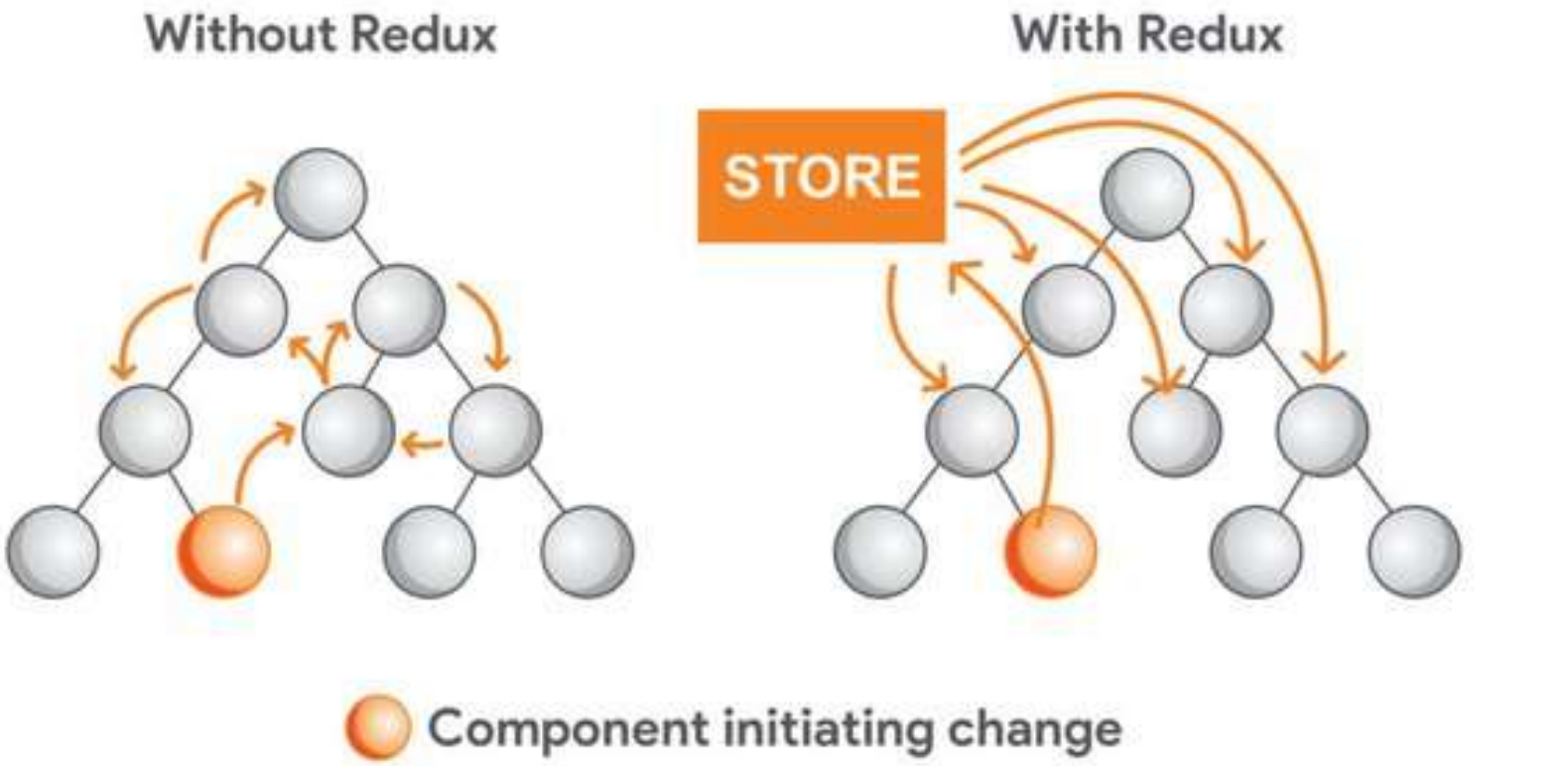
- Redux is an **open-source** JavaScript library for **managing and centralizing application state**. It is most commonly **used with libraries such as React or Angular for building user interfaces.**

- **Developer**-Similar to Facebook's Flux architecture, it was created by Dan Abramov and Andrew Clark.

Let's Understand it with Diagram



React application with and without Redux-



Why Redux is Needed

- In large applications:
- Data is shared between many components
- Managing state becomes complex
- Debugging becomes difficult

Redux solves this by:

- Storing all data in **one central place**
- Making state updates **predictable and controlled**
- Redux helps you manage **"global" state** - state that is needed across many parts of your application.
- Predictable state **updates**.
- Efficient **component communication.**

Limitations of Redux

- **1. Boilerplate Code-**Traditional Redux requires:
  - Actions
  - Reducers
  - Types

  (Though **Redux Toolkit** reduces this)
- **2. Learning Curve-**Concepts like:
  - Reducers
  - Middleware
  - Immutability
    (Can be difficult for beginners)
- **3. Overkill for Small Apps**
- For simple apps, Redux adds unnecessary complexity
- **4. Indirection**
- Logic is separated:
  - UI → action → reducer → store

Comparison with React State & Context API

Redux vs React Built-in State

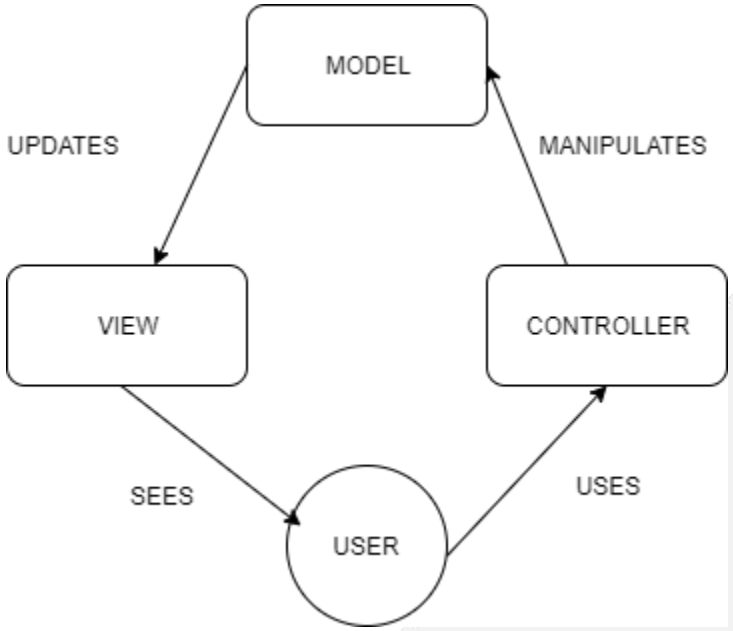
Feature	Redux	React State (useState/useReducer)
Scope	Global	Local
Complexity	High	Low
Best For	Large apps	Small/medium apps
Debugging	Advanced (DevTools)	Limited
Data Sharing	Easy	Hard (prop drilling)

Redux vs Context API

Feature	Redux	Context API
Purpose	Full state management	State sharing
Performance	Optimized (fine control)	Can re-render many components
Scalability	High	Limited in large apps
DevTools	Yes	No advanced tools
Middleware	Yes	No

Redux Architecture-

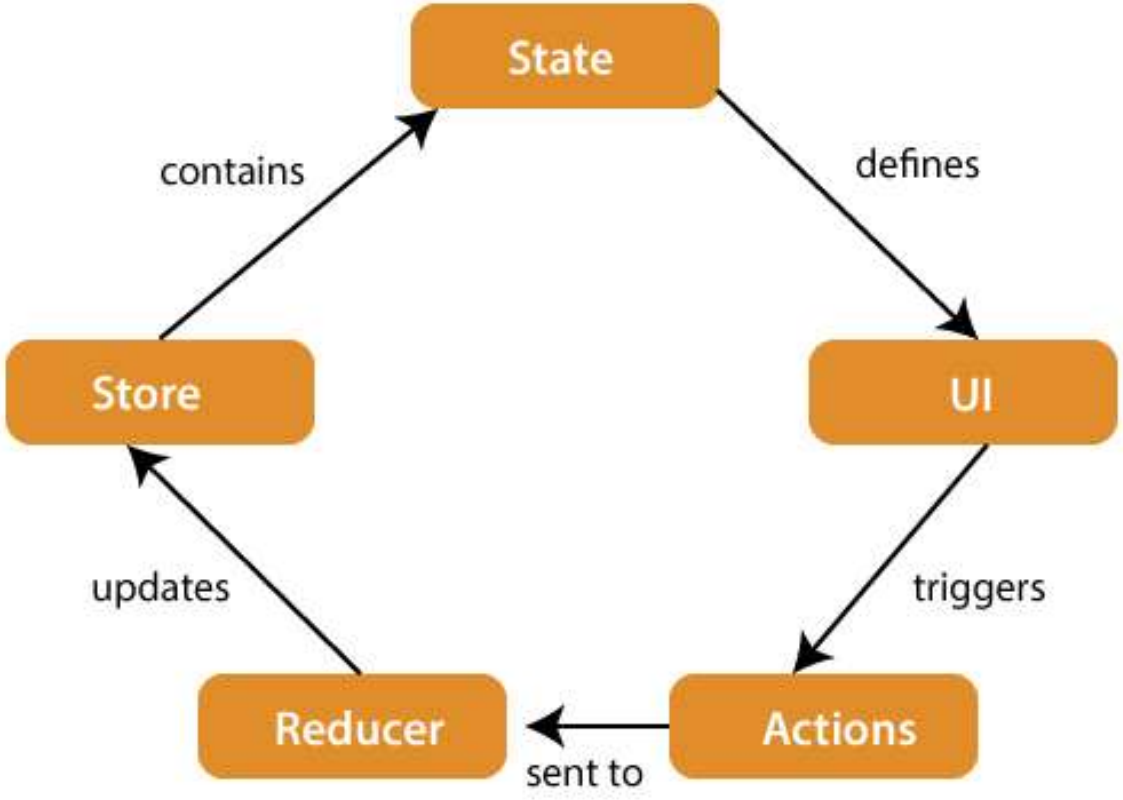
- Redux is a **predictable state container** for JavaScript applications.
- Redux derives its ideas from the **Flux architecture**. It is basically a flux-like approach to React applications which follow the MVC architecture-



- **Local Component UI State** : It is the state which is used only inside a particular component and not being shared across any other components . Example : Enabling or Disabling button. This state can be easily managed using Local React component state and using Redux will be overkill.
- **Shared UI state** : It is the state which is being initialized in one parent component and shared across multiple component which can be direct children or part of different component tree . Example : User authentication data to authorize various component and apply personalization rules . The state has to passed from one component to another until it gets where it is needed .This makes state difficult to maintain and less predictable , Hence we need Redux to manage application state in a centralized place and keep changes in app more predictable and traceable manner.

Redux Architecture- redux architecture with its component.

1. **UI**
2. **Actions**
3. **Reducer**
4. **Store**
5. **State**



Example-banking system

1. Account balance = **state**
2. Bank = **store**
3. (deposit/withdraw) = **action**
4. Bank employee = **reducer**

The components of Redux architecture are explained below.-

- **STORE:** A Store is a place where the entire state of your application lists. It manages the status of the application and has a dispatch(action) function. It is like a brain responsible for all moving parts in Redux.
- **ACTION:** Action is sent or dispatched from the view which are payloads that can be read by Reducers. It is a pure object created to store the information of the user's event. It includes information such as type of action, time of occurrence, location of occurrence, its coordinates, and which state it aims to change.
- **REDUCER:** Reducer read the payloads from the actions and then updates the store via the state accordingly. It is a pure function to return a new state from the initial state.

- React **component** subscribes to the store and get the latest state during initialization of the application.
- To change the state, React component creates necessary **action** and dispatches the action.
- **Reducer** creates a new state based on the action and returns it. Store updates itself with the new state.
- Once the state changes, store sends the **updated state** to all its subscribed component

Redux - Installation-

Basic Redux Installation

Install Redux core:

npm install redux

Install React-Redux (Important)

This connects Redux with React:

npm install react-redux

Redux Toolkit

- Redux Toolkit includes the Redux core, as well as other key packages we feel are essential for building Redux applications (such as Redux Thunk and Reselect).
- It's available as a package on NPM for use with a module bundler or in a Node application:

```
# NPM
npm install @reduxjs/toolkit

# Yarn
yarn add @reduxjs/toolkit
```

Basic Methods & Functions in Redux

No.	Method / Function	Purpose	Syntax
1	**createStore()**	Creates the Redux store (central data container)	const store = createStore(reducer);
2	**getState()**	Returns the current state of the store	store.getState();
3	**dispatch()**	Sends an action to the reducer to update state	store.dispatch({ type: "ACTION_TYPE" });
4	**subscribe()**	Listens for state changes (used to update UI)	store.subscribe(() => { ... });
5	**Reducer Function**	Defines how state changes based on action	function reducer(state, action) { return newState; }

Basic Methods & Functions in Redux

No.	Method / Function	Purpose	Syntax
6	**Action (Object)**	Describes what operation should happen	{ type: "INCREMENT" }
7	**Action Creator**	Function that returns an action	const action = () => ({ type: "INCREMENT" });
8	**combineReducers()**	Combines multiple reducers into one	combineReducers({ user: userReducer })
9	**applyMiddleware()**	Adds middleware (e.g., logging, async handling)	createStore(reducer, applyMiddleware(middleware))
10	**Middleware**	Handles side effects (API calls, logging)	const middleware = store => next => action => { ... }

React-Redux Methods

No.	Method / Hook	Purpose	Syntax
11	**<Provider>**	Makes Redux store available to all components	<Provider store={store}>
12	**useSelector()**	Access state from store in component	const data = useSelector(state => state.value);
13	**useDispatch()**	Dispatch actions from component	const dispatch = useDispatch();

Redux Toolkit

No.	Function	Purpose	Syntax
14	**configureStore()**	Simplified store creation	configureStore({ reducer })
15	**createSlice()**	Creates reducer + actions together	createSlice({ name, initialState, reducers })
16	**createAsyncThunk()**	Handles async operations (API calls)	createAsyncThunk("name", asyncFn)

Basic Project Setup

Step 1: Create a Redux Store

```
import { configureStore } from "@reduxjs/toolkit";

const store = configureStore({
reducer: {}
});

export default store;
```

Step 2: Provide Store to React App

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { Provider } from "react-redux";
import store from "./store";

const root =
ReactDOM.createRoot(document.getElementById("root"));

root.render(
<Provider store={store}>
<App />
</Provider>
);
```

Components of Redux Architecture

1.Store (Central Data Container)-We import createStore from **Redux**
This function is used to **create a central store**

store holds the entire application state

```
import { createStore } from "redux"; //create store is a function
import reducer from "./reducer";

const store = createStore(reducer);
export default store;
```

2.State (Application Data)

State is the data stored inside the store

```
{
count: 0
}
```

3.Action

Action is a plain object describing **what to do**

```
const incrementAction = {
type: "INCREMENT"
};
```

4.Reducer Function

```
function reducer(state = initialState, action)
```

This function:

- Receives **current state**
- Receives **action**
- Returns **new state**
- If no state exists → use default (initialState)

Redux Example Code

```
import { createStore } from "redux";

// Step 1: Create Reducer
function counterReducer(state = { count: 0 }, action) {
  if (action.type === "INCREMENT") {
    return { count: state.count + 1 };
  }
  return state;
}

// Step 2: Create Store
const store = createStore(counterReducer);

// Step 3: Display Initial State
console.log("Initial State:", store.getState());

// Step 4: Dispatch Action
store.dispatch({ type: "INCREMENT" });

// Step 5: Display Updated State
console.log("Updated State:", store.getState());
```

Redux to manage authentication(Architecture)

UI (Login Form / Protected Pages)

↓

dispatch(action)

↓

Redux Slice (authSlice)

↓

Store (auth state)

↓

useSelector (read state)

↓

UI Updates (login/logout)

1. Create Auth Slice

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
isAuthenticated: false,
user: null
};

const authSlice = createSlice({
name: "auth",
initialState,
reducers: {
login: (state, action) => {
state.isAuthenticated = true;
state.user = action.payload;
},
logout: (state) => {
state.isAuthenticated = false;
state.user = null;
}
}
});
```

2. Create Store

```
import { configureStore } from "@reduxjs/toolkit";
import authReducer from "./authSlice";

export const store = configureStore({
reducer: {
auth: authReducer
}
});
```

3. Connect Store to React

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { Provider } from "react-redux";
import { store } from "./store";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
<Provider store={store}>
<App />
</Provider>
);
```

4. App Component (Login/Logout UI)

```
import React from "react";
import { useSelector, useDispatch } from "react-redux";
import { login, logout } from "./authSlice";

function App() {
const { isAuthenticated, user } = useSelector((state) => state.auth);
const dispatch = useDispatch();

return (
<div style={{ textAlign: "center", marginTop: "50px" }}>
<h1>Authentication App</h1>

{isAuthenticated ? (
<>
<h2>Welcome, {user.name}</h2>
<button onClick={() => dispatch(logout())}>Logout</button>
</>
) : (
<>
<h2>Please Login</h2>
<button
onClick={() =>
dispatch(login({ name: "Student" }))
}
>
Login
</button>
</>
)}
</div>
);
}

export default App;
```